VETRA

Site Documentation

Updated October 2026

This document explains what VETRA is, how the codebase is organized, what every page and feature does, how the pieces fit together, and what to do next. It covers the public marketing site, the buyer (customer) app, the vendor app, and the admin console.

This document was originally written when VETRA was a pure front-end prototype (no server, no database, no real authentication) and much of its page-by-page detail below still describes that era accurately — it hasn't been rewritten line-by-line. That's no longer the current state of the project, though: **a real backend now exists in `backend/`, is deployed and live, and most of the frontend calls it for real** — see `BACKEND_GUIDE.md` (build plan, data model, full API reference), `backend/README.md` (how it's actually deployed and what's been verified), and `ADMIN_GUIDE.md` (the live services this runs on, how to reach the database directly, and how to create a Super Admin without the console) for the authoritative, up-to-date picture. §9 below is kept current with exactly what's real vs. still-mock; where the rest of this document says something is "mock," "fake," "no backend," or "`localStorage`-only," check §9 first — some of those have since become real.

---

1. What VETRA is

VETRA is a marketplace for buyers ("customers") and sellers ("vendors") in Nigeria, with VETRA's own delivery riders and an internal **admin console** for VETRA staff. It's a static-file frontend (plain HTML/CSS/JS, no build step) talking to a Node/Express + MySQL backend (`backend/`); both run on one cPanel hosting account, the site at `https://vetrahq.org` and the API at `https://api.vetrahq.org`. The customer app, vendor app, rider app and admin console are all fully wired to it. Buyers pay through Paystack at checkout, and riders and vendors are paid by Paystack transfer (`PAYMENTS.md`). Delivery (the rider app at `rider/`, the admin Deliveries console, the cart's delivery form, live tracking on `customer/orders.html` and the public `track.html`, escrow release) is documented in `DELIVERY_SYSTEM.md`. What's still genuinely unbuilt: buyer↔vendor chat (the dead frontend stub was removed, see §9 point 10). See §9 for the full breakdown.

Numbers you see while using a freshly-migrated database (before any real signups/orders) will legitimately be zero/empty — that's the real backend correctly reporting an empty database, not a bug. Any specific dollar/naira figures or seeded example rows mentioned elsewhere in this document are leftover from the original mock-data era and describe what the *demo* used to show, not live figures.

---

2. Tech stack & conventions

- **No build step, no framework.** Plain HTML files, plain CSS files, plain `<script>` tags. Open any `.html` file directly in a browser and it works.
- **One shared design system per app area.** `customer/assets/style.css`, `vendor/assets/style.css`, and `admin/assets/style.css` each define the *same* base design tokens and layout primitives (sidebar, header, bottom nav, stat cards, tables, modals, forms, switches). This is intentional, established duplication — each area's stylesheet is self-contained so a change in one app doesn't silently break another, at the cost of some repeated CSS between them. The root `style.css` is a separate, simpler design system used only by the public marketing pages (`index.html`, `about.html`, `contact.html`, `signin.html`, `signup.html`).
- **"App shell" pattern.** Every logged-in page (customer, vendor, admin) uses the same skeleton: a collapsible left sidebar on desktop (≥900px), a bottom tab bar on mobile, a dark header with the VETRA logo, and a `<main class="page-shell">` content area. `assets/interactions.js` (customer, vendor) and `assets/ui.js` (admin) wire up the sidebar collapse/expand behavior identically in each area.
- **Shared chrome across all three apps lives in one project-root file, `shared-ui.js` (item 40).** `VetraModal` (`confirm()` / `info()` / `photoPreview()`) and `VetraChrome` (`wireSidebarToggle()` / `wireSidebarCollapse()` / `applyAvatar()`) replace what used to be three separate, drifting reimplementations (`AdminUI` / `VendorUI` / `CustomerUI` each writing their own version) with one, self-mounting its own modal markup on first use rather than depending on it being pasted into each page's HTML. Each app's own `assets/ui.js` is now a thin wrapper around it, so nothing that already called `AdminUI.confirm(...)` etc. needed to change. See §10's design-system reference for the full picture.
- **Mock data lived in JS, historically.** Product lists, orders, and customer/vendor records used to be hard-coded into the HTML or into an `assets/data.js`-style file rather than fetched from an API — essentially all of that has since been replaced with real fetches via `api-client.js` against the real backend, including the vendor app's own KYC/payout/avatar-cover-photo UI, and `admin/assets/data.js` itself has been deleted. See §9 for exactly what (very little) is still genuinely unbuilt.
- **Live chat.** Every page includes the Smartsupp live-chat snippet, with `hideWidget: true` so it only opens when a page's "Support" icon calls `smartsupp('chat:open')` (see `assets/support.js`, shared verbatim between customer and vendor).

---

3. Folder map

```
VETRA/
├── index.html, about.html, contact.html      Public marketing site
├── signin.html, signup.html                  Buyer/Vendor sign-in and sign-up (riders sign in at
rider/)
├── reset-password.html                       Set a new password from an emailed link (also rider
invites)
├── track.html                                Public order tracking by tracking code
(DELIVERY_SYSTEM.md)
├── style.css, signup.css                     Public-site design system
├── script.js, signin.js, signup.js,           Public-site + auth page JS
│   password-toggle.js                       (main.js was deleted — item 24; script.js
│                                             covers index/about/contact, password-toggle.js
│                                             is shared with every app's password fields)
├── api-client.js                             VetraAPI — one fetch wrapper for the real backend,
│                                             used by every app (see §9 point 5 and
BACKEND_GUIDE.md)
├── shared-ui.js                              VetraModal/VetraChrome — one shared confirm/info/
│                                             photo-preview modal + sidebar-toggle implementation
│                                             for all three apps (item 40), each app's own
│                                             assets/ui.js now a thin wrapper around it
├── google-signin.js                          Real "Continue with Google" popup flow (§4)
├── assets/brand/                             Shared logo files (every app references these via
../)
├── assets/banners/                           Dashboard banner images — admin-managed, see item 21
│                                             (banner-1.jpg, banner-2.jpg are the seeded defaults)
│
├── customer/                                 Buyer-facing app
│   ├── dashboard.html, explore.html (category browsing merged in),
│   │   category.html (redirect stub → explore.html, see §5),
│   │   store.html, vendors.html, product.html, cart.html,
│   │   orders.html, notifications.html, settings.html
│   │   (chat.html removed entirely, item 37 — see §9 point 10)
│   └── assets/ (style.css, interactions.js, ui.js, filters.js, orders.js,
│                report-issue.js, products.js, cart-store.js,
│                cart.js, badges.js, vendors.js, purchase-history.js,
│                support.js)
│
├── vendor/                                   Seller-facing app
│   ├── dashboard.html, products.html, orders.html,
```

```
│   │   earnings.html, profile.html, notifications.html
│   └── assets/ (style.css, interactions.js, ui.js, product-actions.js,
│                add-product.js, orders.js, order-tracking.js,
│                reports.js, profile.js, kyc.js, payout.js, support.js)
│
├── rider/                                  VETRA rider app (phone-first, one page)
│   ├── login.html, index.html              Jobs, history, payout bank account, duty switch,
│   │                                        delivery-area picker (see "Rider app" below)
│   └── assets/ (rider.css, rider.js)
│
├── extras/                                 Retired/unused assets, kept but not referenced
│   └── logo.png, logo-mark.png, logo-mark-white.png, *-avatar-dummy.jpg
│
└── admin/                                  Staff-only console
    ├── login.html                          Admin sign-in gate
    ├── dashboard.html                      Platform overview
    ├── customers.html                      Manage buyer accounts (list)
    ├── customer-detail.html                One customer's full profile + activity
    ├── vendors.html                        Manage vendor stores (list)
    ├── vendor-detail.html                  One vendor's full profile + activity
    ├── reports.html                        Reports & disputes queue
    ├── deliveries.html                     Dispatch, riders, delivery rates (DELIVERY_SYSTEM.md)
    ├── payments.html                       Payouts, payments, refunds, commission (Super Admin
only)
    ├── activity.html                       Audit trail / activity log
    ├── settings.html                       Admin profile, team, platform switches
    └── assets/ (style.css, ui.js, session.js, format-helpers.js, dashboard.js,
                 customers.js, customer-detail.js, vendors.js, vendor-detail.js,
                 contact-change.js, reports.js, deliveries.js, payments.js,
                 activity.js, settings.js)
```

4. Public site

Page	Purpose
`index.html`	Landing page — hero, featured categories/products, marketing content.
`about.html`	Brand story / mission.

Page	Purpose
`contact.html`	Contact info / form (static).
`signin.html`	Buyer/Vendor sign-in, toggled by a "Buyer / Vendor" tab. A small "Admin sign in" link at the bottom now routes staff to `admin/login.html`.
`signup.html`	Buyer/Vendor account creation, same toggle pattern as sign-in; supports a `#Vendor` URL hash to land directly on the vendor tab.

How sign-in "works" today: `signin.js` validates only that the visible fields aren't empty, then redirects — Buyer → `customer/dashboard.html`, Vendor → `vendor/dashboard.html`. No password is actually checked and no session is created. `admin/login.html` follows the same mock pattern, redirecting to `admin/dashboard.html`. **This is the single most important thing to replace before going live** — see §9.

---

index.html

- `.buy-now-btn` **(4x, "Popular this week" product cards)** — Each button carries `data-product`, `data-price`, `data-vendor`. Click opens the checkout modal via `openCheckout(btn)` in `script.js:92`. Populates `#checkout-vendor`, `#checkout-product-name`, `#checkout-product-price`, `#checkout-total-price` (price + flat `deliveryFee = 1500`, `script.js:75`), resets the 4 guest-detail fields, then unhides `#checkout-overlay`. State: none persisted, plain DOM.
- `#checkout-confirm-btn` **("Confirm purchase")** — `script.js:125–154`. Validates all 4 fields (`checkout-name`, `checkout-phone`, `checkout-email`, `checkout-address`) are non-empty; email additionally checked against `emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/` (`script.js:90`). Invalid/empty fields get `style.borderColor = '#e0475c'` and focus jumps to the first bad field. On success: button text → "Processing…", disabled, then after a 700ms `setTimeout` swaps `#checkout-step-review` for `#checkout-step-success` and fills `#checkout-success-text` with an interpolated confirmation message. No real order is created (`// TODO: replace with a real checkout/payment API call.`).
- `#checkout-cancel-btn` **/** `#checkout-close-btn` **/ clicking the overlay backdrop /** `Escape` **key** — all call `closeCheckout()` (`script.js:117`), which just sets `checkoutOverlay.hidden = true`.
- `.nav-toggle` **(mobile hamburger)** — `script.js:2–14`. Toggles `.open` class on `.nav-links` and flips `aria-expanded`; clicking any nav link also closes the menu.
- **Smartsupp support widget** — not present on this page as a dedicated support button (index.html has no sidebar); the Smartsupp snippet loads but stays hidden until interacted with globally.

about.html

- No interactive JS-driven elements beyond the shared `.nav-toggle` mobile menu (`script.js:2–14`) — the rest of the page is static marketing content. `script.js`'s FAQ-accordion and checkout-modal code look for elements (`.faq-item`, `#checkout-overlay`) that don't exist on this page, so those code paths simply no-op (`if (!q || !a) return;` etc.).

contact.html

- `.audience-toggle button[data-mode]` **("I'm a buyer" / "I'm a Vendor" tabs)** — `script.js:35-48`. Click toggles `.active` on the clicked button, then shows/hides `#buyer-fields` vs `#Vendor-fields` by setting `style.display = 'grid'` / `'none'`. Pure DOM toggle, no validation change (both field groups' inputs are optional; only `#name`, `#email`, `#message` on the base form carry `required`).
- `#contact-form` **submit ("Send message" button)** — `script.js:51-61`. `e.preventDefault()`, swaps the submit button's text to "Message sent", calls `form.reset()`, then restores the original button text after a `2600` ms `setTimeout`. No network request — the reset happens immediately, so the "sent" state is purely cosmetic feedback and the form fields are already cleared before the label reverts.
- `.faq-item .faq-q` **(4 FAQ questions)** — `script.js:17-32`. Click toggles `.open` on the clicked `.faq-item`; closes any other currently-open item first (single-open accordion), and animates via `el.querySelector('.faq-a').style.maxHeight = a.scrollHeight + 'px'` (or `null` when closing).

signin.html

- `.toggle button[data-mode]` **(Buyer / Vendor tabs)** — `signin.js:8-30`. Toggles `active` / `aria-selected` on buttons, shows/hides `#buyer-form` vs `#Vendor-form` via `.hidden` class, updates `.subtitle` copy, and hides `#guest-continue-wrap` when on the Vendor tab (guest checkout is buyer-only). Also runs on page load if URL has `#Vendor` hash (`signin.js:34-37`), auto-clicking the Vendor tab button — this is how `signup.html`'s post-registration redirect and `index.html`'s "Become a Vendor" links land users on the right tab.
- `.continue-btn` **("Continue")** — **since updated to call the real backend** (`POST /api/auth/signin`, see BACKEND_GUIDE.md's `/api/auth` section) instead of the mock-gate behavior described below, which is kept here as a record of the original design: reads whichever form (`buyer-form`'s `#username` / `#password`, or `Vendor-form`'s `#store-username` / `#Vendor-password`) matches the active tab, redirects to `customer/dashboard.html` / `vendor/dashboard.html` on success, shows the real server's error message (`#signin-error`) on failure.
- `.google-btn` **("Continue with Google")** — **real now** (`google-signin.js` at the project root, `POST /api/auth/google`) — opens a real Google popup, and prompts for phone/address (+ business name for a vendor) afterward if Google didn't already have them on file. See BACKEND_GUIDE.md's `/api/auth` section for the full flow.
- `#guest-continue-link` **("Continue as Guest →")** — plain `<a href="customer/dashboard.html">`, no JS — a direct navigation link (buyer tab only; the whole `#guest-continue-wrap` is hidden on the Vendor tab).
- **Password show/hide toggle** — injected by `password-toggle.js:18-50` on every `input[type=password]` on the page (so both `#password` and `#Vendor-password`). Wraps the input in `.password-field-wrap`, appends a `.password-toggle-btn`; click flips `input.type` between `password` / `text` and swaps the eye/eye-slash SVG icon. Local DOM state only (`input.type`), resets on reload.

signup.html

- `.toggle button[data-mode]` **(Buyer / Vendor tabs)** — `signup.js:7-31` via `activateMode(mode)`. Same pattern as signin.js: toggles active state, shows/hides `#buyer-form` / `#Vendor-form`, updates `.subtitle`.

Runs on load if `location.hash === '#Vendor'` (`signup.js:28–31`) — this is the entry point for `index.html`'s `signup.html#Vendor` links and `vendor-protection.html`'s "Become a Vendor" CTA.

- `.continue-btn` **("Continue") — since updated to call the real backend** (`POST /api/auth/signup`) instead of the mock-gate behavior described below, kept as a record of the original design: same non-empty-field validation on whichever form is active, then a real account is created and the user is redirected to `signin.html` / `signin.html#Vendor` — or the real server's error (`#signup-error`) is shown, e.g. for a duplicate email.
- `.google-btn` — real now, same as signin.html.
- **Password toggle** on `#password` and `#Vendor-password` — same `password-toggle.js` behavior as signin.html.

buyer-protection.html / vendor-protection.html

- No page-specific JS interactions — purely static informational/marketing content (hero, value-grid cards, step timelines, CTA links). Only the shared `script.js` mobile-nav toggle (`.nav-toggle`) applies, and neither page has the FAQ/checkout/contact-toggle elements those code paths target, so those parts of `script.js` no-op. All links (`Start shopping`, `Become a Vendor`, `Talk to support`, etc.) are plain anchor navigation, not JS-driven.

---

5. Customer (buyer) app

All pages live in `customer/` and share both the sidebar and the mobile bottom nav: **Home, Explore, Orders, Profile** — item 20 below added Chat to the bottom nav so the two matched exactly (it used to be sidebar-only), then item 26 removed Chat from both entirely (see that item — the page and its code are still there, just unlinked from anywhere). Cart and Notifications are header icons rather than sidebar items on every page.

Category and Explore used to be two separate, near-duplicate browse pages (same filter panel, same product-grid pattern) — they were merged into one `explore.html`, and Order History and a direct Support entry point were added to the nav in the same pass; see item 17 below for the full write-up.

Page	What it does
`dashboard.html`	Home feed: picture-only promo banner carousel, admin-managed (see item 21 and admin's Settings → Site Banners), a category rail that links into `explore.html` pre-filtered, and a featured product grid — every card opens `product.html` and has a real **Add to Cart**.
`explore.html`	Full product browse grid **and** category browsing in one page: a picture-only, admin-managed promo banner (same one `dashboard.html` shows — see item 22) sits above a row of category quick-filter chips (8, styled like the trending chips above them — no image grid), then the filter panel + product grid, whose own category checkboxes cover the same 8 categories plus Sports — this is where `category.html`'s browsing job now lives. Same real click-through/add-to-cart as dashboard.html.

Page	What it does
`category.html`	**Redirect stub only** — `window.location.replace('explore.html' + location.search)`, so old links/bookmarks (including `category.html?cat=X` deep links) still land correctly. No real page content anymore; see item 17.
`store.html`	A single vendor's storefront (name, products, and a **Reviews** section — see below) — `id="store-name"` is filled in by JS from a query param/mock lookup. Product cards open `product.html` and add to the real cart, same as the other browse pages. Its vendor data now lives in the shared `assets/vendors.js` (item 20), not an inline object local to this page.
`vendors.html`	**New** — a directory of every vendor on VETRA, with a live search-by-name box. Reached from `explore.html`'s "TOP VENDORS" → "See more", which was previously an inert button. See item 20.
`product.html`	**New** — a single product's full detail page (image, price, description, "Sold by" link, quantity + Add to Cart, reviews). See below.
`cart.html`	Real cart contents rendered from `CartStore` — quantity steppers, remove, and an order summary that recomputes from actual line items. No static/dummy data.
`orders.html`	**New** — order history and per-order delivery tracking. Status filter tabs (All/Pending/Processing/Shipped/Out for delivery/Delivered/Cancelled) via `wireOrderFilterTabs`; each order card is also a click-to-expand accordion (`wireOrderExpand`, both in `assets/orders.js`) revealing a step timeline (Placed → Processing → Shipped → Out for delivery → Delivered) that mirrors the escrow flow described on `../buyer-protection.html`. Each order's **Report an issue** button opens a real "File a report" modal (see below) rather than just opening live chat. Now a direct sidebar/bottom-nav item (see item 17) — also still reachable from Settings → Account overview → "View orders" and from the "Order delivered" notification card on `notifications.html`.
`notifications.html`	Real notification feed (`GET /api/notifications`) — order status updates, KYC/account decisions, click-to-read and mark-all-read; each card links to wherever it's about (usually `orders.html`).
`settings.html`	Account & preference toggles (notifications, etc.); the "Orders placed" row under Account overview links to `orders.html`; a **Help & Support** card holds the "Contact support" entry point that used to live in the sidebar/bottom-nav (see item 17).

`customer/assets/interactions.js` wires: sidebar collapse/reopen, the banner carousel's dots, `wireProductCardClicks` (any `[data-product-id]` card opens `product.html?id=<id>`), `wireAddToCartButtons` (adds the real product to `CartStore`, see below), and `updateCartBadge` (keeps every page's header cart icon showing a live item count).

Real cart + product catalog (`assets/products.js`, `assets/cart-store.js`). Closes a gap where "Add to Cart" only ever navigated to `cart.html` with nothing in it. `products.js` is a shared, hard-coded `PRODUCTS` catalog (23 entries, keyed by id) covering every product shown on `dashboard.html` / `explore.html` / `store.html` — the same "Gaming Laptop" on the dashboard and on Godsfavour Laptops' store page is now literally one catalog entry (`vendorId: 'gods-favour'`), not two

coincidentally-matching listings. `cart-store.js`'s `CartStore` persists `{id, qty}` pairs to `localStorage` (key `vetra_customer_cart`) and always re-resolves name/price/image from the catalog live, so a cart line can never drift out of sync with it. Every product card across those pages carries `data-product-id="<id>"`, which is what both the click-to-detail and add-to-cart handlers key off.

Product badges — "New" / "Hot" (`assets/products.js`'s `getProductBadge()`, `assets/badges.js`). Used to be 5 hand-typed `<span class="product-badge hot/new">` elements sitting in `explore.html`'s markup only — the same "Gaming Laptop" showed "New" there but nothing on `dashboard.html`, with no data behind which products got a badge or why (and a "Trending" label that was just alternate text on the same `.hot` CSS class — not a distinct concept in the code). Replaced with real, data-driven logic: every catalog entry in `products.js` now has `createdAt` (an ISO date) and `salesCount` (an int); `getProductBadge(product)` returns `"New"` if `createdAt` is within the last 21 days, else `"Hot"` if `salesCount >= 50`, else nothing — "New" wins if a product happens to qualify for both. `badges.js`'s `decorateProductBadges()` applies that decision to the DOM: for every `[data-product-id]` card, it clears any existing `.product-badge` and (if `getProductBadge()` returned one) prepends a fresh one into the card's thumbnail wrapper. Called on `DOMContentLoaded` for `dashboard.html` / `explore.html` (static card markup) and right after the grid is built for `store.html` (JS-templated cards) — the same product now shows the identical badge everywhere it appears, which the old hardcoded version never guaranteed. See BACKEND_GUIDE.md §3 ("Product badges") for exactly how `createdAt` / `salesCount` should come from real columns/queries once a backend exists — the front-end logic itself doesn't need to change.

Reviews (`store.html`, `product.html`). Each mock vendor in `store.html`'s `VENDORS` object (and each catalog entry in `products.js`) carries a `reviews` array (name, rating, date, text), rendered as an average-score summary plus a review-card list, both tagged "Verified purchase" per the "verified reviews only" claim made elsewhere on the site. A "Write a review" form (star-picker + textarea) prepends a new review in-memory on submit. **The "only after a completed order" claim is now a real, enforced check** (item 25) — `assets/purchase-history.js`'s `PurchaseHistory` (a `localStorage`-backed list of purchased product ids, written by `cart.js`'s checkout handler) decides whether the form even renders: `product.html` checks `hasPurchased(product.id)`, `store.html` checks `hasPurchasedFromVendor(vendorKey)` (true if any purchased product's `vendorId` matches). No purchase → the form stays `hidden` and only the hint text shows ("You can only review a product/vendor after a completed order..."); with one → the form appears with a confirming hint instead. This used to just show a hint claiming the check happened without ever actually running it.

File a report (`orders.html`, `customer/assets/report-issue.js`). Each order's "Report an issue" button now opens a real modal — a reason dropdown, a details textarea, and a note pointing to `../buyer-protection.html`'s escrow-hold explanation — instead of just opening live chat. This was, until item 21, the one place in the customer app that broke the "customer pages never touch admin's `localStorage`" rule (`dashboard.html`'s banner does now too — see item 21): `orders.html` loads `../admin/assets/data.js` directly and calls its new `VetraAdmin.addReport()` function, so a report filed here actually lands in the same `vetra_admin_state_v1` state `admin/reports.html` reads — tested and confirmed to show up there, attributed to the (mock) signed-in buyer "Amaka Obi," with no false "handled by an admin" attribution on the activity-log entry (`logActivity()` gained a `systemEvent` option specifically so a buyer-originated event doesn't get

credited to whichever admin last signed in on that browser). Each order card carries a `data-vendor-id` matching a real `admin/assets/data.js` vendor id where one exists (`Naija Home Essentials` → `v3`) and a synthetic one otherwise (the customer app's own separate `store.html` vendor directory was never unified with admin's vendor list — see §8) — either way the report is created and queued correctly, it just won't deep-link to a real vendor-detail page for the synthetic ids. Submitting swaps the button for a "Report filed" tag so the same order can't be reported twice.

Shared JS loaded on nearly every page: `assets/interactions.js` (sidebar collapse/reopen, banner carousel, global add-to-cart delegation) and `assets/support.js` (Smartsupp live-chat open/close). Both are IIFEs/top-level listeners bound on `DOMContentLoaded`.

Shared interaction reference (applies to every customer page unless noted): - `#header-sidebar-toggle` — `Vetra.wireSidebarToggle()` (`customer/assets/interactions.js:18-37`). Toggles `.hidden-desktop` on `#app-sidebar`, sets inline `display: none` / `''`, and shows `#app-sidebar-reopen` (adds `.show`) when hidden. - `#app-sidebar-reopen` (floating ≡ button) — same function, `interactions.js:30-36`; un-hides the sidebar and hides itself. - `#sidebar-collapse-toggle` (chevron inside sidebar) — `Vetra.wireSidebarCollapse()` (`interactions.js:39-47`); toggles `.collapsed` on `#app-sidebar` (icon-only rail mode). - `.support-btn` — no longer in the sidebar/bottom-nav (see item 17); now a single "Contact support" button in `settings.html`'s Help & Support card. Same global delegated click listener in `customer/assets/support.js:75-80` calls `openLiveChat()` (`support.js:68-73`), which adds `body.livechat-open`, calls `smartsupp('chat:open')`, and lazily injects a `#livechat-close-btn` (`ensureCloseButton()`, `support.js:44-66`) whose click calls `smartsupp('chat:close')` then `window.location.reload()` — the listener itself is unchanged, only where the trigger element lives moved. - `[data-product-id]` **cards** (any product card on dashboard/explore/store) — `Vetra.wireProductCardClicks()` (`interactions.js`); clicking anywhere on the card except a real `<a>` / `<button>` opens `product.html?id=<id>`. - `.add-cart` **/** `.add-btn` (any "Add to Cart" button on any product card, delegated at `document` level) — `Vetra.wireAddToCartButtons()`; calls `CartStore.addItem(productId, 1)` (reading `data-product-id` off the closest card), shows "Added ✓" on the button for 1200ms, and calls `updateCartBadge()`. Falls back to navigating to `cart.html` only if the button's card has no `data-product-id` at all (shouldn't happen on any current page — a defensive fallback, not the normal path).

customer/dashboard.html

- **Banner carousel (`#app-banner`)** — now picture-only (no text/color overlay) and admin-managed rather than 2 hard-coded slides; see item 21. An inline script (loaded after `../admin/assets/data.js`) builds `.banner-slide` / `.dots` markup from `VetraAdmin.getSiteBanners()` on `DOMContentLoaded` — each slide is just an `<img class="banner-slide-img">`, nothing else — falling back to a 2-image `DEFAULT_BANNERS` constant if `VetraAdmin` isn't available or no banners are set, then calls `Vetra.wireBannerCarousel()` itself once the slides exist (the automatic call from `Vetra.init()` runs first and no-ops, since the markup ships empty). Dots — `Vetra.wireBannerCarousel()`. Click on a dot calls `goTo(index)`, swapping `.active` between `.banner-slide` elements and their matching dot; also auto-advances every `4000ms` via `setInterval`.

- **"See more" (`.cat-head .see-more`)** — plain `<button>`, no `id`/handler anywhere in the codebase; currently inert (does nothing on click).
- **Category rail (`.cat-card`, 5 tiles)** — click navigates to `explore.html?cat=<name>`, which pre-filters that page's grid to the matching category (or shows an honest "no products yet" if this demo has none in it). Previously an unlinked, inert `<div>` — see §8. Pointed at `category.html` until item 17 merged that page into `explore.html`.
- **Product cards (6, `[data-product-id]`)** — click anywhere on the card (except a real link/button) opens `product.html?id=<id>` (`Vetra.wireProductCardClicks()`); the **Add to Cart** button adds the real product to `CartStore` — see the shared reference above and §8's fix write-up.
- **Header icon links** (`notifications.html`, `cart.html`, `settings.html` via avatar) — plain `<a>` navigation, not JS. The cart icon additionally shows a live `.cart-count-badge` when the cart is non-empty.

customer/explore.html

As of item 17, this page carries everything the old `category.html` used to, in addition to its own original content — there is no longer a separate category-browsing page.

- **Category quick-filter chips (`.category-chip`, 8 buttons)** — plain `<button>`s reusing the `.trending-chip` pill styling (same look as the "Power bank / Wireless earbuds / …" row above them), not an image grid — `category.html`'s old `.categories-grid` / `.category-card` tile grid was removed in favor of this, per direct feedback that it should be simple buttons instead. Each button's `data-category` is read on click and passed to `filterByCategory(name)` (an export from `initProductFilters()`, `filters.js`); since the filter panel now has a matching `[data-filter-category]` checkbox for all 8 of these categories (plus Sports), every chip click resolves through the normal "check the box, re-run `applyFilters()`" path — including a category with zero matching products in this page's grid, which correctly ends up at 0 visible results and the filter panel's own generic empty-state text (no category-specific wording needed anymore, since there's no longer a case where a clicked category *lacks* a filter-panel checkbox). Also reads a `?cat=<name>` query param on page load and calls the same function, so `dashboard.html`'s category rail (and `category.html`'s redirect) can deep-link in pre-filtered.
- **Filter button `#exploreFilterBtn`** — `initProductFilters()` in `customer/assets/filters.js`, invoked with `{ triggerSelector: '#exploreFilterBtn', panelId: 'exploreFilterPanel', gridSelector: '#app-products .grid' }` (inline script at the bottom of the page). Click toggles `.open` on `#exploreFilterPanel` (`filters.js:78-80`); clicking anywhere outside the panel/button while open closes it (`filters.js:82-86`, delegated document click listener).
- **Filter panel controls** (`#exploreSortSelect`, category checkboxes `[data-filter-category]` — now 9 categories: the 8 quick-filter chips' categories (Appliances, Phones & Tablets, Health & Beauty, Home & Office, Electronics, Fashion, Computing, Supermarket) plus Sports, so every chip and every product's category has a matching checkbox — price checkboxes `[data-filter-price]`) — each `change` event runs `applyFilters()` (`filters.js:28-76`):
- Reads checked `[data-filter-category]` values and checked `[data-filter-price]` values (each price checkbox's `value` is a `"min-max"` string, e.g. `"0-20000"`, `"150000-0"` where `0` upper bound means unbounded — `filters.js:44-50`).

- A card matches if `catMatch && priceMatch` where `catMatch` requires `card.dataset.category` to be in the selected set (or no categories selected = match-all), and `priceMatch` requires `card.dataset.price` (parsed `Number`) to fall in at least one selected bucket.
- Non-matching cards get `style.display = 'none'`.
- Sorting: `sortValue` from `#exploreSortSelect` — `price-asc` / `price-desc` sort numerically on `data-price`; `name-asc` uses `data-name.localeCompare`; `featured` preserves original DOM order. Sorted cards are re-appended into the grid (`sorted.forEach((card) => grid.appendChild(card))`).
- Updates `filterBtn`'s `.active` class (non-zero selection count) and the `[data-filter-count]` text ("Showing X of Y" / "Showing all N").
- Creates a `.filter-empty-state` div once, lazily (`ensureEmptyState()`, `filters.js:19-26`), shown/hidden based on `visibleCount`.
- `[data-filter-clear]` **("Clear filters")** — `filters.js:92-102`. Unchecks every checkbox, resets the sort `<select>` to `"featured"`, re-runs `applyFilters()`.
- **Banner (`#app-banner`)** — picture-only and admin-managed, identical setup to `dashboard.html`'s (same inline script, same `../admin/assets/data.js` load, same `VetraAdmin.getSiteBanners()` call) — see item 22. Previously had its own hard-coded text+image slides with different copy ("Explore" / "Featured Vendors") from `dashboard.html`'s; now both pages render the exact same admin-controlled image list, so there's one banner to manage, not two to keep in sync by hand.
- **Product cards (12, `[data-product-id]`)** — the original 6 plus `category.html`'s 6 (Premium Laptop Stand, Wireless Headset, USB-C Cable, Phone Stand, Power Bank, Screen Protector), re-rendered onto this page's own `.product` / `.product-img` card markup (they used `.product-card` / `.product-thumb` on the old page — same catalog entries via `products.js`, just re-templated to match this grid's existing card style). Click → `product.html?id=<id>`; **Add to Cart** → real `CartStore.addItem()`.
- **Vendor mini-cards (`#top-vendors-rail`)** — an inline script fetches the real vendor list (`VetraVendorDirectory.load()`, `GET /api/vendors`) on `DOMContentLoaded` and renders each `.vendor-mini-card` from it; the whole rail (including its "TOP VENDORS" header) stays `hidden` rather than rendering an empty section if the fetch fails or nothing qualifies. **As of item 41, "Top Vendors" means KYC-verified vendors only** — the list is filtered to `v.kyc_verified` before being sliced to the first 8, so an unverified store is never surfaced here as a highlighted pick even if it would otherwise have made the cut. The store name is escaped (`VetraAPI.escapeHtml`) before being templated in, closing a prior unescaped-name gap. `.vendor-mini-name` also picked up a name-overflow fix in the same commit (item 41): it used to have `overflow` / `text-overflow: ellipsis` on an unconstrained `inline-flex` element, so those rules never actually engaged and a long store name could overlap the neighboring card; it's now split into a width-constrained row plus an inner truncating span (`min-width: 0` is what lets it actually shrink instead of overflowing).
- **"See more" (Top Vendors, `.vendor-rail .see-more`)** — a plain `<a href="vendors.html">`, not JS. Used to be an inert `<button>`; see item 20. (`vendors.html`'s own directory grid, described below, is not filtered to verified-only — that distinction is specific to this rail.)
- **Trending chips (`.trending-chip`)** — plain `<a href="#">`, no handler.

customer/vendors.html — new, see item 20

Reached from `explore.html`'s "TOP VENDORS" → "See more". All logic is an inline `<script>` at the bottom of the page.

- **Vendor grid (`#vendorDirectoryGrid`)** — rendered from `getAllVendors()` (`assets/vendors.js`) via a `.map().join('')` template on page load; each card (`.vendor-card`) shows avatar, name, a `.status-pill` (`verified` / `pending`, reusing the same pill styling `vendor/orders.html`'s shipment-status pills use), category, rating, and location, and links to `store.html?vendor=<id>`.
- `#vendorSearchInput` — `input` event re-renders the grid, filtering `getAllVendors()` client-side by `vendor.name.toLowerCase().includes(query)`. No backend, so this is the same in-memory list every render, not a real search endpoint. An empty result set hides the grid and shows `#vendorDirectoryEmpty` with the literal typed query echoed back (`No vendors match "<query>".`) rather than silently showing nothing.
- `#vendorDirectoryCount` — "`N` vendors on VETRA" with no active search, or "Showing `X` of `N` vendors" while a search query narrows the list.
- `#vendors-back-btn` — same referrer-based `history.back()` / fallback-to-`explore.html` pattern as `store.html`'s and `product.html`'s back buttons.

customer/category.html — now a redirect stub, see item 17

No longer a real page. Its `<head>` fires `window.location.replace('explore.html' + window.location.search)` immediately (plus a `<meta http-equiv="refresh">` fallback and a plain link in the body, in case JS is disabled). This exists only so old links/bookmarks — including `category.html?cat=Electronics`-style deep links — still land somewhere correct; every internal link that used to point at `category.html` was updated to point at `explore.html` directly instead (so the redirect hop isn't actually hit from anywhere in the app itself anymore).

customer/product.html — new

`?id=<id>` looks the product up in `products.js`'s shared `PRODUCTS` catalog (`getProduct(id)`); an unknown/missing id shows a `#product-not-found` empty state with a link back to Explore instead of a blank page. Renders image, category, name, price, location (if the catalog entry has one), a "Sold by `<vendor>`" link (only shown when `vendorId` is set — not every product has one), and the description.

- **Quantity stepper (`#pd-qty-decrement` / `#pd-qty-increment`) — as of item 41, this is cart-aware, not a plain unbounded counter.** It caps at the product's real stock minus however much of that same product is already sitting in `CartStore` (`CartStore.getQty()`), not just the raw stock figure — repeatedly clicking Add to Cart across page visits used to be able to silently exceed real availability, since only total stock was ever checked, never what the shopper had already added. Hitting the cap flips `#pd-add-to-cart` to a disabled "Out of Stock" button immediately, rather than only finding out at checkout.
- `#pd-add-to-cart` **("Add to Cart")** — `CartStore.addItem(product.id, qty)`, then `Vetra.updateCartBadge()`, then the same "Added ✓" / disabled-for-1200ms feedback pattern the card buttons use. Deliberately **not** given the `.add-cart` class the card buttons use — see §8's bug write-up for why sharing that class here broke the button entirely.

- **Reviews section** — byte-for-byte the same pattern as `store.html`'s (star-input, `#review-form`, `renderReviewSummary()` / `renderReviewList()`), just scoped to `product.reviews` instead of `vendor.reviews`. Same in-memory-only caveat: a submitted review is lost on reload. **The form itself is now gated on a real purchase check (item 25)** — `#review-form` ships with the `hidden` attribute and only loses it if `PurchaseHistory.hasPurchased(product.id)` is true on page load; `#review-form-hint` swaps its text between the locked/unlocked cases. Not just visually hidden — no rating/text controls exist to submit against until the check passes.
- **Back button (`#product-back-btn`)** — same referrer-based `history.back()` / fallback-to-`explore.html` pattern as `store.html`'s back button.

customer/store.html

Most logic lives in an inline `<script>` block at the bottom of the page. Vendor "database" is the shared `VENDORS` object in `assets/vendors.js` (item 20 — previously a hard-coded object local to this file only) keyed by `gods-favour`, `daniel-ice-fish`, `jennet-jeans`, `me-n-u-chops`; `vendors.html`'s directory reads the same object via that file's `getAllVendors()`.

- **Page load** — reads `?vendor=<id>` from the URL (`new URLSearchParams(window.location.search).get('vendor')`), resolves it to `vendorKey` (falling back to `'gods-favour'` if missing/unknown) and looks that up in `VENDORS`. Populates `#store-avatar`, `#store-name`, `#store-bio`, `#store-rating`, `#store-orders`, `#store-response-time`, `#store-location`, `#store-member-since`, sets `#store-cover`'s `background-image` (vendor's `cover` image or `coverGradient` CSS string), sets `#store-category` text + toggles `.pending` class if `vendor.status === 'pending'`, and injects a verified-badge SVG into `#store-verified-badge` only if `vendor.status === 'verified'`. `vendorKey` (not just the resolved `vendor` object) is kept around because the review-gate (below) needs the id, not just its data.
- The phone/call button that used to sit here (`#store-call-btn`, an `alert('Calling...')` placeholder) and the "Message Vendor" button before it are both gone now — see items 26-27. `.store-header`'s action row is just the vendor name/badge/category now; nothing in it is a placeholder pretending to work.
- `#store-back-btn` **(chevron "back" button)** — `store.html:282-290`. If `document.referrer` exists and shares the page's origin, calls `window.history.back()`; otherwise redirects to `explore.html`.
- **Products grid (`#storeProductsGrid`)** — rendered from `vendor.products` array via a `.map().join('')` template; each product object now also carries an `id` matching an entry in `products.js`'s shared `PRODUCTS` catalog (e.g. `gods-favour-wireless-mouse`), rendered onto the card as `data-product-id`. Click → `product.html?id=<id>`; each card's "Add to cart" button (`.add-btn`) → the same real `CartStore.addItem()` path every other page's cards use.
- **Star rating input (`#star-input .star-btn`, 5 buttons)** — `setStars(value)` (`store.html:370-375`) sets `starInput.dataset.value` and toggles `.active` on every button whose `data-star` is `<=` the clicked value (so clicking star 3 highlights 1-2-3). Local DOM state only (`starInput.dataset.value`), reset to `0` after each successful submission.
- **Review-form visibility (real purchase gate, item 25)** — before any of the submit logic below runs, `#review-form` ships `hidden` and only loses it if `PurchaseHistory.hasPurchasedFromVendor(vendorKey)`

(`assets/purchase-history.js`) is true — i.e. the buyer has actually checked out with a product whose `vendorId` matches this store. `#review-form-hint`'s text swaps between "You can only review a vendor after a completed order with them" (locked) and a confirming line (unlocked). No purchase, no form at all — this replaced a version that always showed the form and just had a comment admitting it couldn't check.

- `#review-form` **submit ("Submit review")** — only reachable once the gate above has un-hidden the form. Reads `rating = Number(starInput.dataset.value)` and `text = document.getElementById('review-text').value.trim()`. Validation: `if (!rating) alert('Please select a star rating before submitting.')` and returns; `if (!text) { textEl.focus(); return; }` (empty text silently blocks submit, no message). On success: `reviews.unshift({ name: 'You', rating, date: new Date().toISOString().slice(0,10), text })` — prepends to the in-memory `reviews` array (a `.slice()` copy of `vendor.reviews`, `store.html:314`), then re-renders both `renderReviewSummary()` (average score + star count) and `renderReviewList()` (every review — including the new one — tagged "Verified purchase," which is at least now backed by a real check on whether the form was reachable at all, even though the tag itself is still applied unconditionally to every review in the array, old seed data included). Resets the star input and clears the textarea. State is **entirely in-memory** (the `reviews` array closure variable) — reloading the page loses the new review, though the purchase itself (and therefore the form staying unlocked) persists via `PurchaseHistory`.

customer/cart.html

Completely rebuilt on top of `CartStore` — no more static/dummy `.cart-item` rows. `renderCart()` reads `CartStore.getItems()` on every change and either shows `#cart-empty-state` (genuinely empty cart — a "Your cart is empty" message + a link to Explore) or `#cart-content` (the real line items + summary), toggled via `hidden` rather than both existing in the DOM with one hand-waved as a placeholder.

- **Per-item row** — image + name link to `product.html?id=<id>`; a "Visit store" link appears only when the product has a `vendorId`. Quantity `−` / `+` (`data-action="decrement"` / `"increment"`) call `CartStore.setQty(id, newQty)` — `setQty` itself removes the line entirely once qty reaches `0`, so decrementing from `1` behaves the same as Remove. **Remove** (`data-action="remove"`) → `CartStore.removeItem(id)`. All three re-render the whole cart and call `Vetra.updateCartBadge()`.
- **Order summary** — `subtotal` is `CartStore.getSubtotal()` (sum of `product.price * qty` across real items, always current); **Delivery** is the sum of each store's quote from `GET /api/delivery/quote` for the chosen state ("Free" for pickup or under free delivery); **Processing fee** is `processingFee(subtotal + delivery)` from `api-client.js`, the amount that covers Paystack's cut; **Total** is all three. The server recomputes every one of these; the cart's numbers are display only.
- **"Save for later" — ✅ real now.** Moves every current cart line into `SavedForLaterStore` (same `localStorage`-backed shape as `CartStore`, a second instance from the same factory — see §9) and clears the active cart. A "Saved for later" section appears on this same page whenever it's non-empty, each line with its own "Move to cart"/"Remove".
- **"Pay securely" — real Paystack checkout.** Requires a buyer account (guests go to sign-up, keeping their cart). Validates the delivery form, then makes one `POST /api/payments/checkout` call for the whole cart, every store's items grouped by vendor, with a per-attempt idempotency key (`sessionStorage`), and sends

the browser to Paystack's checkout page. Paystack returns the buyer to `orders.html?payment=<reference>`; the orders are created on the server only once the payment is confirmed (`PAYMENTS.md` §2). A mixed cart is one payment but one order per store.
- Dropped from this rebuild: the old per-item delivery/pickup radio buttons and the "Contact vendor" button that opened the generic chat list — neither was backed by real per-item data; "Visit store" (above) is the real-data replacement where a vendor link makes sense.

customer/orders.html

`customer/assets/orders.js` renders the buyer's real orders from `GET /api/orders/mine`; `customer/assets/report-issue.js` files reports.

- **Returning from Paystack.** On `?payment=<reference>`, `handlePaymentReturn()` shows "Confirming your payment with Paystack…", polls `GET /api/payments/<reference>` every 3 seconds for up to a minute (that route confirms with Paystack itself if the webhook hasn't arrived), then: **Payment received** with the new order codes (and the cart is emptied); **Payment refunded** if it arrived after the checkout expired; **Still confirming** if Paystack hasn't answered yet; or **Payment not completed** with a link back to the saved cart. The `?payment=` parameter is removed so a refresh doesn't replay it.
- **Status filter tabs** — All, Pending, Processing, Shipped, Out for delivery, Completed, Cancelled; filter the cards client-side by `data-status`.
- **Order cards** — thumbnail, order ID (the `VTA-` tracking code), items, store and verified badge, amount (goods + delivery), status pill. Clicking expands the card: a step timeline, the delivery panel (`GET /api/delivery/orders/:id`: the buyer's delivery or pickup code, the rider's name and phone once assigned, the rider's live position on a map while en route, missed attempts, the event timeline), the item list (an item the vendor marked unavailable says it was refunded), the delivery fee, and any refund ("₦X refunded to your card").
- **"I received it, all good"** appears on a delivered order whose escrow is still held, with the auto-release deadline. It calls `POST /api/delivery/orders/:id/confirm`, which releases escrow and pays the vendor.
- **"Report an issue"** opens a modal (reason + details) that files a real report against the vendor (`POST /api/reports`); an open report holds the order's escrow until an admin resolves it.
- Panels for orders in motion refresh every 30 seconds.

customer/chat.html — deleted, see item 37

This page no longer exists. It was unlinked from the rest of the app (item 26, below) well before it was actually removed — a buyer↔vendor messaging UI, entirely inline-`<script>`, running on hardcoded mock conversations (`chatData`, 4 fictional vendors) that was never migrated to the real backend like every other customer page was, and had zero real entry points left pointing to it. Deleted outright this session, markup/script/mock-data and its dedicated CSS block in `customer/assets/style.css` — see §9 point 10. Buyer↔vendor messaging remains a genuinely unbuilt feature (no schema, no routes, no real-time layer), it just no longer has a dead frontend stub pretending otherwise.

customer/notifications.html

- **"Order delivered" card** — the only notification wrapped in an `<a href="orders.html">` (with `unread` class); plain navigation, no JS. `orders.html` also has its own sidebar/bottom-nav item and a Settings link now (item 17), so this is one of three entry points rather than the only one.
- **"Mark all read" (`.page-link` button)** — no `id`, no JS handler anywhere in the codebase; inert.
- Other notification cards ("Flash sale", "Payment confirmed") are plain non-interactive `<div>`s.

customer/settings.html

All interaction logic is an inline `<script>` at the bottom of the page (no separate JS file); also loads `assets/ui.js` (`CustomerUI`) and `../password-toggle.js` for the 3 password fields in the Security card.

Profile card — per-field inline edit (item 31). Each field (Full name, Phone number, Email address, Delivery address) starts as plain read-only text (`.field-value`) with its own small pencil button (`.field-edit-btn`), not a single "Edit" toggle for the whole card. Clicking a field's pencil swaps that one field's `.field-view` for its `.field-edit` row (a real input plus a green check / red x pair) — every other field stays locked. Confirm (`wireSettingsProfileFields()`) writes the trimmed value back to `.field-value`; Enter confirms, Escape cancels, clicking the x reverts to the last-committed value. No backend yet — each field's `// TODO: replace with a real per-field save API call` comment marks where a `PATCH` per field would go.

- **Profile photo** — `#settings-avatar-btn` ("Change photo") opens the device file picker; picking a file opens the shared photo-preview popup (`CustomerUI.photoPreview()`, see "The shared confirm/photo-preview modals" below) with a large circular preview and full-size Save/Cancel buttons, replacing what used to be two icon-only buttons crowded onto the avatar's corner. Saving converts the file to a data URL, writes it to `localStorage` (`vetra_customer_profile_avatar`), updates the avatar on this page, and calls `Vetra.applyCurrentCustomerAvatar()` (`assets/interactions.js`) so the small avatar icon in every page's header updates immediately too — see item 36 for why that header sync needed fixing.
- `#password-edit-toggle` **("Edit", Security card)** — `enterPasswordEditMode()`. Unlike the Profile card above, the Security card still uses the whole-form show/hide pattern (not per-field) since a password change isn't "editing a current value" the way a name or email is — there's no existing password to display. `#password-form` carries `.hidden` by default and only 2 fields (`#new-password`, `#confirm-password` — `#current-password` was removed, see item 19), and clicking Edit removes `.hidden` from the form, adds it to the edit-toggle button, and focuses `#new-password`.
- `#password-cancel-btn` — `exitPasswordEditMode()`: `form.reset()`, then re-adds `.hidden` to the form and re-shows the edit-toggle button — the whole form disappears again, not just locks.
- `#password-form` **submit ("Update password")** — No validation of match/strength/length despite being a password-change form. On submit: `e.preventDefault()`, `form.reset()`, `exitPasswordEditMode()` — no actual password update occurs (`// TODO: wire up to real update-password endpoint`).
- `#manage-payment-btn` **("Manage payment methods")** — click → `CustomerUI.info({...})`, an in-app popup, not a native `alert()` (see item 34).
- `#deactivate-account-btn` **("Deactivate account") — ✅ real.** `CustomerUI.confirm({...danger: true...})`; on confirm, `PATCH /api/auth/deactivate` (sets `status = 'suspended'` on the real account),

then clears the session and redirects to sign-in.

- **"View orders →" link** (Account overview card) — plain `<a href="orders.html">`, not JS.
- **"Contact support" (Help & Support card)** — a plain `<button class="support-btn">`; not a JS handler local to this page, it's caught by the same global delegated listener described in the shared interaction reference above (`support.js`). This card is where the sidebar/bottom-nav "Support" entry moved to (item 17) — it's the only `.support-btn` left anywhere in the customer app.
- **"Log out"** — plain `<a href="../signin.html">`, not JS (no session to actually clear).
- **Notification toggles (`.toggle-switch input[type=checkbox]`, 3 rows)** — plain checkboxes, no JS handler; purely visual on/off state that doesn't persist or do anything.
- **Password reveal toggles** — injected by `../password-toggle.js` on all 3 password fields, same mechanism as signin.html.

The shared confirm/photo-preview modals (`customer/assets/ui.js`, `CustomerUI`). Mirrors `admin/assets/ui.js`'s `AdminUI` one-to-one (see §7's "The shared confirm modal"), added this session so the customer app stopped falling back to the browser's own `confirm()` / `alert()` dialogs (the ones that show "This page says…") for its danger-zone actions, payment-methods placeholder, star-rating validation on both review forms, and the cart's checkout/save-for-later placeholders — see item 34. `CustomerUI.confirm(opts)` / `.info(opts)` work exactly like `AdminUI`'s; `CustomerUI.photoPreview(opts)` is new (not in `AdminUI`, since admin's avatar upload saves instantly with no confirm step at all) — see item 36. **As of item 40, no page markup is needed for this anymore.** `CustomerUI` / `VendorUI` / `AdminUI` are now thin wrappers delegating to a project-root `shared-ui.js` (`VetraModal`), which builds its own `#confirm-modal` / `#photo-preview-modal` DOM and appends it to `document.body` the first time any page calls `.confirm()` / `.info()` / `.photoPreview()`, rather than expecting each page to have pasted the same `.modal-overlay` / `.modal-panel` block in before its `<script>` tags. `product.html`, `store.html`, `settings.html`, and `cart.html` no longer carry that markup at all — see item 40.

---

6. Vendor (seller) app

All pages live in `vendor/` and share the sidebar: **Dashboard, Products, Orders, Earnings, Profile**.

Page	What it does
`dashboard.html`	Real store KPIs (revenue, orders, active listings — no "store views" card; that was a fabricated metric with no view-tracking behind it, dropped rather than left fake), a real recent-orders list, and a "My Products" preview grid. Has the **Add Product** button, which opens a modal form.
`products.html`	Full product grid with **Edit** / **Remove** actions per card, category filter tabs above the grid, and a category label on every card (`vendor/assets/product-actions.js`).

Page	What it does
`orders.html`	Order list with status filter tabs (Pending/Processing/Shipped/Out for delivery/Delivered/Cancelled) via `wireOrderFilterTabs` in `orders.js`. Each order has an **Update shipment** button opening a modal (`assets/order-tracking.js`) to set shipment status, carrier, and tracking number — updates that order row's status pill and tracking line in place. Below the order list, a **Reports against your store** panel (read-only — see below) shows reports filed against this vendor.
`earnings.html`	Revenue breakdown / earnings history, plus a **Payout Account** section (see below) above the payout history list.
`profile.html`	A store background/cover photo and avatar (both editable via a popup — see below), a **Store Details** card where each field (name, owner, email, phone, address, bio) is independently edited via its own pencil button rather than one "Edit Profile" unlocking all six at once, a **Business Verification (KYC)** section (see below), notification toggles, security section (change password, 2FA, sign out), and a **Danger Zone** (deactivate store / delete account).
`notifications.html`	Real vendor-side notification feed (`GET /api/notifications`) — new orders, shipment-relevant events, KYC decisions, and account/store status changes, click-to-read and mark-all-read.

Add Product modal (`vendor/assets/add-product.js`): a full form — name, category (one of 13 site-wide categories, see item 35), price, a stock quantity stepper, description, up to 4 image upload slots with live preview, and one optional video upload slot — real images upload via `POST /api/uploads` (Cloudinary), and the product itself is created for real via `POST /api/products`, so it survives a reload and shows up for real customers browsing the catalog, not just prepended to the in-page grid. If `products.html`'s category filter tabs are active on a category other than "All," a newly-added card that doesn't match the active tab is hidden immediately after being added (`window.VetraProductFilter.reapply()`), rather than appearing regardless of the filter.

Reports against your store (`orders.html`, `vendor/assets/reports.js`). A vendor can see reports filed against their store but can't resolve or dismiss them — that stays admin-only (`admin/reports.html`), keeping one place where a report is actually closed out. What a vendor *can* do is respond: an open report's **Submit evidence** button opens a modal (a response textarea plus a mocked photo-upload slot borrowed from the Add Product modal's media-upload UI) — submitting it swaps the button for an "Evidence submitted — awaiting review" tag on that card. This is deliberately a read-only mirror of the same report data admin sees, not a second copy of the moderation queue.

Business Verification / KYC (`profile.html`, `vendor/assets/kyc.js`). Closes a gap where the site's buyer-facing copy claimed "every Vendor passes ID and business verification before their first listing ever goes live" (`buyer-protection.html`) with no actual UI for a vendor to do that verification. The form takes a CAC registration number plus two uploads — a valid ID and a CAC certificate (image preview for photos, a filename chip for a PDF) — reusing the Add Product modal's upload-slot pattern. Submitting requires all three fields.

The panel actually closes after submitting (item 33) — it used to just gray out and stay on-screen. Submitting swaps the whole card from the editable form to a read-only summary (`#kyc-summary`, `applyKycStatus()`): CAC number, submitted date **and time**, and both file names, plus a status badge and a

note explaining what's next. Four states, driven by a `{status, cacNumber, idFileName, cacFileName, submittedAt, rejectionReason}` object persisted to `localStorage` (`vetra_vendor_kyc_state`, so — unlike before — the closed/pending view survives a reload instead of resetting to "Not submitted" every time): - `not_submitted` — the form is open and editable (the original, unchanged default). - `pending` — form hidden, summary shown, badge "Pending review." - `verified` — form hidden, summary shown, badge "Verified," note explaining buyers can now see the Verified Vendor badge. - `rejected` — the form **reopens automatically**, pre-filled with the last CAC number, alongside the summary (so the vendor can see what they submitted and why it was rejected) and a red "Resubmission needed" badge; the submit button relabels to "Resubmit for verification."

Like the rest of this app, it's a page-local mock with no persistence to the admin console — reloading resets nothing anymore (the point of the fix above), but it still does **not** write into `admin/assets/data.js`'s `localStorage`, and nothing on this page can ever *reach* the `rejected` state on its own, since only a real admin decision can do that and the two apps deliberately don't share state (see `vendor/assets/kyc.js`'s own header comment for exactly why, and `BACKEND_GUIDE.md`'s new KYC section — §3 and §5 — for the real API that replaces this `localStorage` object with a genuine per-vendor status an admin's decision actually reaches). The admin-side review (`admin/vendor-detail.html`'s "Business Verification (KYC)" card, described below) is separate seed data representing documents an admin has already reviewed, not something this form actually submits to.

Payout Account (`earnings.html`, `vendor/assets/payout.js`). Closes a gap where `vendor-protection.html` describes payouts running "straight to the bank account on file" and this same page already captioned every payout-history row "Payout to bank account" — with no UI anywhere to actually set one, on either the vendor form or the admin data model. The new section (above Payout History) takes a bank, a 10-digit account number, and an account name; saving validates the account number's length, then swaps to a read-only view showing the account masked (`•••• 6789`) with an **Edit Account** button. Page-local mock, like KYC — nothing persists past a reload, and (deliberately, for a real version) even this demo never shows the full account number again once saved, only the masked form, modeling what a real implementation's display rule should be even though the underlying mock data is trivial to inspect.

Shared JS: `assets/interactions.js` (`VetraUI`, sidebar toggle/collapse only — no banner carousel or add-to-cart delegation, since the vendor app has neither banners nor a public product grid) and `assets/support.js` (identical Smartsupp open/close logic to the customer copy, but targets `#header-support-btn` on vendor pages, which carries the `.support-btn` class the same delegated listener matches).

Shared reference: - `#header-sidebar-toggle` / `#app-sidebar-reopen` / `#sidebar-collapse-toggle` — `VetraUI.wireSidebarToggle()` / `wireSidebarCollapse()` (`vendor/assets/interactions.js:15-44`), functionally identical to the customer version. - `#header-support-btn` **(header icon)** — opens Smartsupp via the same `openLiveChat()` / `ensureCloseButton()` pattern in `vendor/assets/support.js:68-80`.

vendor/dashboard.html

- `#add-product-btn` **("Add Product")** — `wireAddProductButton()` in `vendor/assets/product-actions.js:34-40`; calls `window.VetraAddProduct.open()` (`vendor/assets/add-product.js:228-234`), which stores `document.activeElement` for focus restoration, un-hides `#add-product-modal`, locks body scroll, focuses `#ap-name`.
- **Add Product modal** — see full breakdown under "Add Product modal (shared)" below.
- **Product grid actions (`.edit-btn` / `.remove-btn`, delegated on `.vendor-products-grid`)** — `wireVendorProductActions()` (`product-actions.js`). Reads the product name from the closest `.vendor-product-card .product-name`. `edit-btn` → `VendorUI.info({...})` (no real edit page exists). `remove-btn` → `VendorUI.confirm({danger: true, ...})`; if confirmed, `card.remove()` — removes the DOM node only (no backend delete, so it reappears on reload). Both go through the shared in-app modal (`assets/ui.js`, `VendorUI`) rather than a native `confirm()`/`alert()` — see item 34.
- **Category filter tabs (`#product-filter-tabs .filter-tab[data-filter]`, `products.html` only)** — `wireProductFilterTabs()` (`product-actions.js`). Shows/hides each `.vendor-product-card` by comparing the active tab's `data-filter` against the card's own `data-category` — see item 35. Exposes `window.VetraProductFilter.reapply()` so `add-product.js` can re-run the active filter immediately after prepending a newly-added card.
- **"See all" links** (Recent Orders → `orders.html`, My Products → `products.html`) — plain `<a>`, not JS.

vendor/products.html

- `#add-product-btn` — identical wiring to dashboard.html (`product-actions.js` + `add-product.js`, same modal markup duplicated in this page's HTML).
- `.edit-btn` / `.remove-btn` — identical `wireVendorProductActions()` behavior as dashboard.html, operating on this page's own `.vendor-products-grid` (9 static product cards).

Add Product modal (shared markup/JS across dashboard.html and products.html) — `vendor/assets/add-product.js`, module `VetraAddProduct`: - **Form fields**: `#ap-name` (text, `required`), `#ap-category` (`<select>`, `required`, one of 8 fixed categories), `#ap-price` (`number`, `min="0" step="1"`, `required`), `#ap-stock` (`number`, `min="0" step="1"`, `required`, default `0`), `#ap-description` (`<textarea>`, `required`). - **Stock stepper (`#ap-stock-decrement` / `#ap-stock-increment`)** — `wireStockStepper()` (`add-product.js:120-133`); `step(delta)` clamps to `Math.max(min, current + delta)` where `min = Number(stockInput.min) || 0`. - **Image upload slots (4x, `#ap-image-grid .media-upload-slot[data-slot-index]`)** — `wireImageSlots()` (`add-product.js:108-118`); each slot's hidden `<input type="file">` `change` event calls `setImagePreview(index, file)` (`add-product.js:42-87`), which creates an object URL (`URL.createObjectURL(file)`), injects an `<img class="media-preview-img">` into the slot, adds a per-slot remove (`×`) button that clears the file and revokes the object URL, and toggles `.has-media` on the slot. - **Video upload slot (`#ap-video-slot` / `#ap-video-input` / `#ap-video-remove`)** — `wireVideoSlot()` (`add-product.js:135-155`); same object-URL preview pattern via `setVideoPreview(file)` (`add-product.js:89-106`), shows the filename in `.video-file-name`. - **Form submit ("Add Product")** — `handleSubmit(e)` (`add-product.js:196-226`). Calls native `form.reportValidity()` first (enforces all `required` fields + number min/step) and returns early if invalid. Then checks `imageFiles.some(Boolean)` — if no image was ever

attached, scrolls to the first image slot and `alert('Add at least one product photo before saving.')`, blocking submission (this check exists *in addition to* the native validity check, since the file inputs themselves aren't marked `required`). On success: builds a new card via `buildProductCard({ name, price, stock, imageUrl })` (`add-product.js:176-194`) — `imageUrl` is a **fresh** `URL.createObjectURL()` call on the first attached image file (kept independent of the form's own preview URLs, which get revoked on close) — HTML-escapes `name` via `escapeHtml()` (`add-product.js:30-37`) to prevent injection, computes the stock-pill via `stockPillMarkup(stock)` (`add-product.js:170-174`, "low" styling when `stock === 0 || stock <= 5`), then `grid.prepend(card)` onto whichever `.vendor-products-grid` exists on the current page, and scrolls the new card into view. Finally calls `close()`. - `close()` (`add-product.js:236-242`) — hides the modal, unlocks scroll, calls `resetForm()` (clears all inputs/previews and revokes every object URL), restores focus to `lastFocusedEl`. - **Close paths**: `#add-product-close`, `#add-product-cancel`, clicking the overlay backdrop, or `Escape` — all call `close()`. - **State**: entirely in-memory (closure variables `imageFiles`, `videoFile`, etc., and the DOM card itself) — a newly added product disappears on page reload, exactly like Edit/Remove.

vendor/orders.html

Loads `assets/orders.js`, `assets/order-tracking.js`, `assets/reports.js` in that order (no shared "orders" state object between them beyond the DOM itself).

- **Status filter tabs (`#order-filter-tabs .filter-tab[data-filter]`)** — `wireOrderFilterTabs()` (`vendor/assets/orders.js`), functionally similar to the customer orders.js version (targets `.order-item` rows inside `#order-list`, not `.order-card`, comparing against `row.dataset.status`) but also caps how many *matching* rows are visible at once — see the next bullet — since this page's version now does pagination, not just show/hide.
- **"Show more" (`#orders-show-more-btn`)** — requested directly ("no more than 5 orders should show unless the show more button is clicked"). `ORDERS_MAX_VISIBLE = 5` in `orders.js`; `applyVisibility()` filters the 9 static `.order-item` rows down to whichever match the active tab, then shows only the first `ORDERS_MAX_VISIBLE` of *those* (not the first 5 of the unfiltered list) and hides the rest, updating the button's label to `Show N more orders` (`hidden` entirely once nothing's left to reveal). Clicking it sets an `expanded` flag and re-runs `applyVisibility()` with no cap. Switching filter tabs resets `expanded` back to `false`, so paging back to "All" after expanding it returns to a capped 5 rather than staying expanded — each filter starts capped. State is a closure variable, like the rest of this page's interactions; resets on reload.
- **"Update shipment" (`.order-manage-btn`, per order row)** — `vendor/assets/order-tracking.js`. Delegated click on `#order-list` (`order-tracking.js:55-59`) finds the closest `.order-item` and calls `openModal(row)` (`order-tracking.js:39-47`): sets `#shipment-order-id` from the row's `.order-id` text, pre-fills `#shipment-status` from `row.dataset.status` (default `"pending"`), `#shipment-carrier` / `#shipment-tracking` from `row.dataset.carrier` / `row.dataset.tracking`, un-hides `#shipment-modal`, locks scroll.
- `#shipment-form` **submit ("Save update")** — `order-tracking.js:71-108`. No required-field validation (carrier/tracking are optional per the form's own label). On submit: writes `status` / `carrier` / `tracking` back onto `activeRow.dataset.*`, updates the row's `.status-pill` class to `status-pill ${status}` and its text via the `STATUS_LABEL` map (`order-tracking.js:21-28` — note `completed` displays as **"delivered"**,

matching the customer-facing wording), and either creates or removes a `.order-tracking-line` element depending on whether `carrier` / `tracking` are set (inserted right before `.order-manage-btn` via `insertBefore`). All 6 status values map 1:1 with customer/orders.js's filter tabs (`pending`, `processing`, `shipped`, `out-for-delivery`, `completed`, `cancelled`).

- **Close paths**: `#shipment-modal-close`, `#shipment-cancel-btn`, backdrop click, `Escape` — call `closeModal()` (`order-tracking.js:49-53`).
- **State**: DOM-only (`row.dataset.*` + the row's own child elements) — resets on reload, no cross-page sync back to `customer/orders.js`'s mock data.
- **"Submit evidence" (`[data-action="evidence"]`, per open report card in the "Reports against your store" panel)** — `vendor/assets/reports.js`. Delegated click on `#report-list` (`reports.js:37-42`) finds the closest `.report-card` and opens `#evidence-modal` (`reports.js:23-29`), clearing/focusing `#evidence-text`.
- `#evidence-form` **submit ("Submit")** — `reports.js:53-66`. `#evidence-text` textarea is `required` (native validation only — no length/content check). On submit, replaces that card's entire `.report-actions` innerHTML with `<span class="badge dismissed evidence-sent-tag">Evidence submitted — awaiting review</span>` — a vendor cannot resolve/dismiss the report itself (that stays admin-only via `admin/reports.html`); this is purely a "vendor responded" marker, DOM-only, resets on reload.
- **Close paths**: `#evidence-modal-close`, `#evidence-cancel-btn`, backdrop click, `Escape` — `closeModal()` (`reports.js:31-35`).
- Report data itself (`#report-list .report-card`, e.g. `rep-1`, `rep-2`) is static seed markup in `orders.html`, **not** pulled from `admin/assets/data.js`'s `localStorage` state — this panel is a page-local mock mirror, unlike `customer/orders.html`'s report-filing which does write into admin's real state.

vendor/earnings.html

`vendor/assets/payout.js` — Payout Account section (`GET` / `PUT /api/vendors/me/payout-account`, see BACKEND_GUIDE.md §5); it's where automatic payouts are sent. `vendor/assets/earnings.js` — the four stat cards (**Total Sales**, **Paid Out**, **Pending Payout**, **VETRA Commission**) and the history list, all from `GET /api/vendors/me/earnings`: each order shows its sale amount, the commission taken and its rate, the net payout, and where that payout is (in escrow, scheduled, sending, paid with the date, waiting for a bank account, retrying, or refunded). Orders from before online payments show as "not paid online" with no payout. See `PAYMENTS.md`.

- **Bank field (`#payout-bank`) — a custom searchable dropdown, not a native `<input list>` / `<datalist>` anymore (item 40).** The native `<datalist>` this used originally was unreliable on mobile browsers (several don't render its suggestion list on a text input at all), so it was replaced with a small hand-built combobox in `vendor/assets/payout.js`: typing filters a rendered `.bank-search-result` button list live (capped to 40 matches), arrow keys move a `.active` highlight through it, Enter/click picks one, and Escape closes it. Picking a result fills the visible text and the same `name → { name, code }` lookup map `loadBanks()` always built, so everything downstream of that (the bank-code lookup, the resolve-on-input flow described below) is unchanged, only the input widget itself was swapped.

- **Account name is resolved, not typed.** The moment a real bank (an exact match in the lookup map) and a 10-digit account number are both present, `scheduleResolve()` debounces ~500ms and calls `GET /me/payout-account/resolve?accountNumber=&bankCode=` — the real, bank-registered name comes back and renders into `#payout-resolved-name`; a bad pair shows Paystack's own error message there instead. The **Save** button stays `disabled` until a resolution for the *current* bank+number pair succeeds (edit either field again and it re-locks) — there is no text input for the account name anywhere on this form anymore.
- `#payout-account-form` **submit ("Save Payout Account")** — sends `{ bankName, bankCode, accountNumber }` (no name field) to `PUT /me/payout-account`, which re-resolves the name server-side before writing anything, so what's saved is always what Paystack actually returned. On success, `renderSavedView()` shows the masked number (`•••• 1234`, server-derived — the real number is never sent back after the initial save) and the verified name, and flips `#payout-status-badge` to "Account on file."
- `#payout-edit-btn` / `#payout-cancel-btn` — same shape as before: editing pre-fills the bank field from what's saved (the account number can't be pre-filled — the server never returns it in full — so it's always re-typed and re-resolved on an edit); Cancel discards an in-progress edit and restores the last-saved view.
- **State**: real, persisted server-side, survives a reload and follows the vendor to any device — see the note above in §9 for the fuller "what used to be `localStorage`-only" history.

vendor/profile.html

Three JS files: `vendor/assets/profile.js` (cover/avatar photo, per-field store-details, security, danger zone, sign-out), `vendor/assets/kyc.js` (Business Verification section), and `vendor/assets/ui.js` (`VendorUI` — the shared confirm/info/photo-preview modals).

Cover photo + avatar (item 31, redesigned in item 36). `#cover-edit-btn` ("Change background") and `#avatar-edit-btn` (the small pencil badge on the avatar) both open a real device file picker. Picking a file opens `VendorUI.photoPreview()` — a popup with a large preview (circular for the avatar, wide for the cover banner) and full-size Save/Cancel buttons — instead of applying the change silently or via the tiny icon-only check/x buttons this used to show inline on the photo itself (too small to comfortably hit; replaced this session). Saving converts the file to a data URL, writes it to `localStorage` (`vetra_vendor_profile_avatar` / `vetra_vendor_profile_cover`), updates the image on this page, and — for the avatar — calls `VetraUI.applyCurrentVendorAvatar()` (`assets/interactions.js`) so the small avatar icon in every vendor page's header updates immediately too, not just this page (item 36; before this fix the header kept showing the old photo everywhere except right after a save on this exact page). The cover has no header equivalent to sync. Neither photo has a real upload endpoint yet — both are `localStorage`-only, page-local to this browser.

Store Details — per-field inline edit (item 32). Each field (Store Name, Owner Name, Email, Phone, Store Address, Store Bio) starts as plain read-only text (`.field-value`) with its own small pencil button (`.field-edit-btn`) — there is no single "Edit Profile" button unlocking all six at once anymore. Clicking a field's pencil swaps that one field's `.field-view` for its `.field-edit` row (a real input/textarea plus a green check / red x pair, via `wireStoreDetailsFields()`) — every other field stays locked, so accidentally leaving one field mid-edit can't also leave five others editable. Confirm writes the trimmed value back to `.field-value`, and — for Store Name/Owner Name specifically — also updates the summary card's `#profile-store-name` / `#profile-owner-name` text immediately. Enter confirms, Escape cancels. No backend yet — each field's `// TODO:`

`replace with a real per-field save API call` comment marks where a `PATCH` per field would go, same convention as `customer/settings.html`'s identical pattern (see §5) and `admin/settings.html`'s Account Details card (see §7).

Business Verification (KYC) form (`#kyc-form`, `vendor/assets/kyc.js`). Fields: ID type (**NIN**, **BVN** or **Driver's licence**), ID number, a **date of birth** field that appears only for a driver's licence (sent for the lookup, never stored), the CAC registration number, and two upload slots (the ID, and the CAC certificate as an image or PDF). Submitting uploads both files to Cloudinary, saves the submission (`POST /api/vendors/me/kyc`), then runs automatic verification through IDCheck.ng (`POST /api/vendors/me/kyc/verify`): the NIN, BVN or licence is matched against the vendor's legal name on file, and the CAC number is looked up. Both passing approves the vendor on the spot; anything else goes to manual review in the admin console. - `#kyc-form` **submit ("Submit for verification" / "Resubmit for verification")** — **Validation**: `cacNumberInput.value.trim()` non-empty **and** both `slots.id.file` and `slots.cac.file` must be freshly attached — resubmitting after a rejection requires re-uploading both documents again, the stored filenames from the previous submission are only shown in the read-only summary, not treated as still-valid uploads. Failure sets `#kyc-note` text to `"Enter your CAC number and upload both documents before submitting."` in red and returns. - **The panel now genuinely closes after submitting instead of just graying out (item 33).** On success, the whole card swaps from the editable form to a read-only summary (`#kyc-summary`, `applyKycStatus()`) showing the CAC number, submitted date **and time**, and both file names — see the KYC writeup earlier in this section for the full `not_submitted` / `pending` / `verified` / `rejected` state machine, which this page and section both implement identically. - **State**: real, in the `vendor_kyc` table, read back with `GET /api/vendors/me/kyc` on every load, the same row the admin console reviews. - `.profile-badge.status-rejected` (new CSS, `vendor/assets/style.css`) — the red badge style shown when `kyc.status === "rejected"`, matching `.status-not-submitted` / `.status-pending`'s existing pattern. - `#change-password-btn` — ✅ **real.** `wireChangePasswordButton()` opens a real modal (current/new/confirm password), submits to `PATCH /api/auth/password` — the same requires-current-password check `customer/settings.html`'s Security card uses. - `#deactivate-store-btn` — ✅ **real.** `wireDangerZoneButtons()`; `VendorUI.confirm({danger: true, ...})`, then on confirm `PATCH /api/auth/deactivate` (sets `status = 'suspended'`), clears the session, and redirects to sign-in. - `#delete-account-btn` — ✅ **real.** Same function; `VendorUI.confirm({danger: true, ...})`, then `POST /api/auth/delete-account` — scrubs the account's PII, sets `status = 'deleted'`, and delists every one of this vendor's products, all in one transaction — before clearing the session and redirecting to sign-in. - `#sign-out-btn` — `wireSignOutButton()`; `VendorUI.confirm({...})`, and if confirmed, `window.location.href = '../signin.html'` (an actual redirect, unlike the other danger-zone buttons which just show an info popup). - **2FA toggle — ✅ real now.** `wireMy2FAToggle()` PATCHes `users.two_factor_enabled` via `PATCH /api/auth/me`; turning it on means the *next* sign-in emails a 6-digit code before issuing a session — see BACKEND_GUIDE.md's "Two-factor authentication" entry and `signin.js`'s code-entry step. The other 4 notification toggle switches (`.switch input[type=checkbox]`) are still plain checkboxes with no backing column — clicking one now honestly reverts and says "not wired up yet" (`wireStubToggles()`) rather than silently doing nothing while looking saved.

The shared confirm/info/photo-preview modals (`vendor/assets/ui.js`, `VendorUI`). Added this session (item 34), mirroring `admin/assets/ui.js`'s `AdminUI` so the vendor app stopped falling back to the browser's

own `confirm()` / `alert()` dialogs for every danger-zone action, the Add Product modal's "photo required" check, and product Edit/Remove. `VendorUI.confirm(opts)` / `.info(opts)` work exactly like `AdminUI`'s; `.photoPreview(opts)` is new (see item 36 above) and not present in `AdminUI`, since admin's avatar upload saves instantly with no confirm step. **As of item 40,** `dashboard.html`**,** `products.html`**, and** `profile.html` **no longer carry a** `#confirm-modal` **/** `#photo-preview-modal` **markup block at all** — `VendorUI` now delegates to the shared, project-root `shared-ui.js` (`VetraModal`), which builds and appends that markup to `document.body` itself the first time any page calls `.confirm()` / `.info()` / `.photoPreview()`, rather than expecting every page to paste in its own copy — see item 40.

vendor/notifications.html

- **"Mark all read" (** `.page-link` **button)** — no `id`/handler in the codebase; inert, same as the customer equivalent.
- All 4 notification cards are static, non-interactive `<div>`s (no links, no click handlers) — this is the one notifications page across both apps where even the "top" notification isn't wrapped in a navigable link.
- Only shared sidebar/support/header behavior applies (`vendor/assets/interactions.js`, `vendor/assets/support.js`).

---

7. Admin console — new in this session

Why it was built

The request was for a place where VETRA staff can monitor customers and vendors platform-wide and suspend accounts, plus "other things that should be on the admin section." The console lives entirely under `admin/`, reuses the exact same app-shell/design system as the vendor app (so it feels native to the site, not bolted on), and — unlike the rest of the prototype — **persists its state to** `localStorage` under the key `vetra_admin_state_v1`, because moderation actions (suspending someone, resolving a report) need to survive a page reload to feel real. `admin/assets/data.js` is the single place that owns this state; every admin page reads and writes through its functions (`VetraAdmin.*`) instead of touching `localStorage` directly, so swapping in a real backend later means changing one file.

Getting in — and "who's logged in"

`admin/login.html` — a staff-only sign-in screen, styled like `signin.html` but with no Buyer/Vendor toggle. It's linked from the bottom of `signin.html` ("Vetra staff? Admin sign in"). Like the rest of the site's auth, it doesn't check a real password — but it does do one real thing: it looks the email you typed up against the admin team roster in `admin/assets/data.js` and, if it matches, tells the data layer **which admin is now "logged in"** (`VetraAdmin.setCurrentAdminByEmail()`). That simulated session is what makes the rest of this section possible — every action taken anywhere in the console is attributed to whoever is currently "logged in," and the console's own role check (Super Admin vs. Moderator vs. Support) reads from it too. The page itself shows a generic "restricted access" subtitle and plain email/password fields — it doesn't print the seeded

demo accounts or note that any password works; you'd need to check `admin/assets/data.js`'s team roster to know which emails match. See §9 for what a real version needs (this is still just an email lookup, not a password check).

Passwords, photos, and "who did what"

Three things were added on top of the original monitor/suspend request, all in direct response to follow-up asks in this session:

- **Reset Password (customers & vendors).** Every customer/vendor row (on `customers.html` / `vendors.html`) and every detail page has a **Reset Password** button. Confirming it generates a temporary password and shows it to the admin in a "share this with them securely" panel — there's no real account system to actually change a password against, so this is as far as a backend-less prototype can go, but the action is logged to that account's activity history like any other.
- **Admin profile photos.** Settings → Admin Profile has a real, working **Change Photo** upload (not a placeholder): it reads the chosen image with `FileReader`, stores it as a base64 data URL on that admin's team record, and immediately reflects it in the profile card, the header avatar on every admin page, and the Admin Team list. This works because the admin console already persists to `localStorage`, unlike the rest of the prototype.
- **Attribution — "which admin attended to a case."** Every activity log entry now records who performed it (`actorId` / `actorName`), and every report gets an `attendedBy` / `attendedAt` pair the moment it's resolved, dismissed, or resolved-via-suspend. That attribution shows up everywhere the relevant record appears — the activity feed ("by Adaeze Umeh"), a report card ("Attended by Adaeze Umeh"), and a customer/vendor's own activity history — regardless of which admin is viewing it. Case-level attribution is intentionally visible to *every* admin; it's the console-wide activity *browsing* below that's role-gated.

Pages

`dashboard.html` **— Platform overview.** Six KPI cards (Total Customers, Total Vendors, Platform Orders, Platform Revenue, Suspended Accounts, Open Reports), a **Quick Actions** row linking straight into a pre-filtered view of the other pages, a **Recent Activity** preview (last 6 audit-log entries), and a **Pending Vendor Applications** table where a new store's application can be **Approved** or **Rejected** right from the dashboard.

`customers.html` **— Manage buyers.** A searchable, filterable table of every customer (name/email, join date, order count, lifetime spend, status). Each row has **View** and either **Suspend** or **Reactivate** depending on current status. Suspending asks for an optional reason via a confirm modal — the reason is stored in the activity log so there's a record of *why*.

`customer-detail.html?id=<id>` **— One customer, in full.** Clicking a customer's name or **View** opens their complete profile instead of a quick-look popup: contact info (phone, delivery address), how and when they signed up, last login, lifetime order/spend stats, a **Recent Orders** list (item, vendor, amount, status pill, carrier/tracking line where shipped — `VetraAdmin.getOrdersForCustomer()`, hidden entirely if empty), **any reports filed against their account** (pulled from the Reports queue), and their **entire activity history** — every suspend/reactivate action ever taken on this specific account, not just the platform-wide feed.

Suspend/Reactivate works from here too, so a moderator reviewing an account doesn't have to bounce back to the list to act on what they just read.

`vendors.html` **— Manage stores.** Same shape as Customers, plus a **Pending Approval** state: new vendor applications show **Approve**/**Reject** instead of Suspend until an admin acts on them. Filter tabs: All / Active / Pending Approval / Suspended.

`vendor-detail.html?id=<id>` **— One store, in full.** Same idea as the customer detail page: store contact info, category, a short description, owner, join date, last login, products/orders/revenue stats, a **Recent Orders** list scoped to this vendor (`VetraAdmin.getOrdersForVendor()`, same rendering as the customer version, listing the buyer instead of the vendor per row), any reports filed against the store, and its complete activity history. The action buttons adapt to status — **Approve/Reject** while pending, **Suspend/Reactivate** once live. A **Business Verification (KYC)** card shows the vendor's submitted CAC number, submission **date and time** (`VetraAdmin.formatDateTime()`, item 33 — this used to show only the date), and the actual ID/CAC document images as clickable thumbnails (`admin/assets/data.js`'s per-vendor `kyc` object — status one of `not_submitted` / `pending` / `verified` / `rejected`) — an admin can open either document full-size in a new tab before deciding, not just see a filename (an earlier version of this card only showed the filename string, which meant "Verify Documents" had nothing to actually verify against; see §8 for the fix); a `pending` submission gets **Verify Documents**/**Reject** actions through the shared confirm modal (`VetraAdmin.setVendorKycStatus()`, which as of item 33 also saves the rejection reason onto `kyc.rejectionReason` — previously only written to the activity log, not the record itself, so re-opening this page later couldn't show *why* a rejection happened, only that one did), logged to that vendor's activity history like any other action. Once reviewed, the card shows "Reviewed `<date and time>`" and, for a rejection, the reason right alongside it. This is independent of the account-level Approve/Reject status — a vendor can in principle be `active` with KYC still `pending`, since the two aren't currently forced to move together (see §8 for why).

`reports.html` **— Reports & disputes.** A moderation queue of flagged accounts, listings, and buyer↔vendor disputes (e.g. "item didn't match description," a chargeback dispute, a suspicious listing). Each open report can be **Marked Resolved**, **Dismissed**, or — if it's against a customer or vendor — resolved with a one-click **Suspend Account**, which suspends the account *and* marks the report resolved in a single confirmation.

`activity.html` **— Activity log / audit trail.** Every suspend, reactivate, approve, reject, resolve, dismiss, password reset, invite, and admin-removal action performed anywhere in the admin console is appended here automatically, newest first, with a relative timestamp ("2h ago"), a type tag, and — for anything an admin actually did — **who did it** ("by Adaeze Umeh"). Filterable by Account Actions / Vendor Approvals / Reports / Orders / Logins. This is what makes the console feel like a real moderation tool rather than a set of disconnected switches — every destructive action leaves a trace.

Visibility here depends on role: a **Super Admin** sees everyone's activity and gets a **"filter by admin"** dropdown to narrow it to one person; a **Moderator or Support** admin only sees platform events (orders, customer/vendor logins — nothing admin-specific) plus their *own* actions, with a note explaining that Super Admins can see more. This is enforced by `VetraAdmin.getVisibleActivity()`, which every activity view (this page and the

dashboard's preview) calls instead of the raw log — but, like every other role check in this prototype, it's a client-side filter, not a real permission boundary (see §9).

`settings.html` **— Admin profile & platform controls.** - **My Profile**: reflects whoever actually signed in (name, email, role badge), with a working **Change Photo** upload. - **Account Details — new, item 32.** A separate card right below My Profile, since admin previously had no way to edit its own name/email at all (unlike vendor's Store Details or the customer Profile card). Full Name and Email each show as plain read-only text with their own pencil button, matching the exact per-field pattern vendor/customer use — clicking one unlocks just that field, confirming writes through `VetraAdmin.updateTeamMemberProfile(me.id, {...})` (new function, same mutate → `save()` → `logActivity("account", ...)` shape every other mutating function in `data.js` follows) so the change shows up correctly in the Admin Team list below too, not just this card. - **Platform Controls**: five real toggles, all backed by `GET` / `PATCH /api/admin/settings` (see §9) — "require approval for new vendors" (**on by default**; off means a new store goes live immediately instead of sitting in Pending), "require ID/business verification for new vendors" (**on by default** — blocks approving a pending vendor without a verified KYC submission), "auto-flag suspicious listings" (**on by default** — scans new/edited listings for restricted terms and auto-files a report if one matches), "allow guest checkout" (**on by default**), "maintenance mode" (**off by default** — blocks new signups and checkout while browsing stays up). - **Site Banners — new, see items 21-22.** A real, working image list, not a cosmetic switch: each row is a thumbnail + Up/Down (disabled at either end of the list) + **Remove** (through the shared confirm modal), plus a **+ Add banner image** button that opens a file picker, reads the chosen image via `FileReader` as a base64 data URL (same pattern as the admin avatar/Change Photo upload above — no real file storage exists yet), and appends it. This list — order and all — is exactly what both `customer/dashboard.html`'s and `customer/explore.html`'s promo carousels render, picture-only, with changes here showing up on both pages on their next load; see `VetraAdmin.getSiteBanners()` / `addSiteBanner()` / `removeSiteBanner()` / `moveSiteBanner()` in `admin/assets/data.js`. - **Admin Team**: the full list of who has console access and their role (Super Admin / Moderator / Support), each with their photo if they've set one, and a **Remove** action that goes through the shared confirm modal. Removing the platform's *last* Super Admin is blocked with an explanation, so the console can't be locked with zero full-access admins. - **Add Admin — invite + email verification.** Clicking **Add Admin** doesn't add someone immediately: it collects name/email/role and creates a *pending invitation*, then opens a **Verify Email** step. Since this prototype has no mail server to actually deliver anything, the code that "would have been emailed" is shown right in the UI, and — per how this was asked for — **entering anything in the verification field completes it**; this is a simulated verification step to demonstrate the invite→verify shape of a real onboarding flow, not a real code check (see `VetraAdmin.verifyTeamInvite()` in `admin/assets/data.js`, and §9). Pending invitations appear in their own list with **Verify**/**Cancel** actions until completed. - **Notifications** and **Security** sections matching the vendor profile page's pattern (password, 2FA, sign out) — the 2FA toggle here is real too (`PATCH /auth/me`'s `twoFactorEnabled`), same as vendor's — see §9. - **Data & Danger Zone** → **Reset Demo Data**: wipes the admin console's `localStorage` state and reloads the original seed data (team roster and any pending invites included). This exists *specifically because* this is a prototype with no real backend — it's how you get back to a clean demo state after clicking around, and it would not exist in a production build.

`deliveries.html` — **Dispatch, riders and delivery rates.** Documented in `DELIVERY_SYSTEM.md` §7: the dispatch queue with assign, cancel and send-back actions and each parcel's timeline; the Riders tab (add, edit, suspend, resend invite; **Mark paid** only for jobs from before online payments); delivery settings and per-state rates (Super Admin).

`payments.html` — **Payments (Super Admin only).** The sidebar link only appears for Super Admins, and every route behind it refuses other roles. Stat cards for the Paystack balance, paid out, waiting to pay and failed payouts; **Payout settings** (the commission % and the **Automatic payouts** switch, which asks for confirmation before turning on); and three tabs: **Payouts** (filter by Failed / No bank account / Scheduled / Paid, with **Retry**), **Payments in** (each checkout: buyer, orders, amount charged, processing fee, channel) and **Refunds** (a form to refund an order by ID, full or partial, with a reason, and the refund history with **Retry** on failed ones after checking Paystack). See `PAYMENTS.md`.

The dashboard also shows a **Paystack Balance** card to Super Admins only.

Rider app (`rider/`)

A one-page, phone-first app for VETRA riders (`rider/index.html` , `rider/assets/rider.js`). Riders never sign up; an admin adds them and they set a password from an emailed link, then sign in at `rider/login.html` .

- **Duty card:** the on/off duty switch (can't go off duty mid-job), the **Delivery area** picker (the state they take jobs in; locked while they hold any job or offer), and today's earnings, earnings waiting to be paid out, and deliveries completed.
- **Jobs tab:** each job shows exactly its next step (accept or decline an offer, enter the vendor's pickup code, start the delivery, enter the buyer's code with an optional photo, or report a missed attempt), with Call and Directions buttons. Polls every 20 seconds and shares the phone's location every 30 seconds while a job is live.
- **History tab:** completed and returned jobs with the payout and its status (Paid, Sending, Pending, Add your bank, Retrying).
- **Bank tab:** the payout account. Pick the bank, type the 10-digit number, the account name comes back from the bank, save. A red dot shows on the tab until one is saved. The rider is paid by Paystack transfer the moment the buyer's code is accepted.

Full detail in `DELIVERY_SYSTEM.md` §8.

The shared confirm modal

Every destructive or state-changing action (suspend, approve, reject, resolve, dismiss, reset demo data) goes through **one reusable confirm dialog** (`AdminUI.confirm()` in `admin/assets/ui.js`), not a bare `window.confirm()` . It shows exactly what's about to happen, names the target by name, optionally collects a reason, and only then calls into `VetraAdmin.*` to make the change. This was a deliberate choice: an admin console where "Suspend" fires immediately on click, with no confirmation, is a foot-gun.

What "add other things that should be on the admin section" became

Beyond the explicitly requested monitoring + suspend features, this session added: 1. **Vendor approval workflow** (pending → active/rejected) — without this, every new vendor signup would go live unmoderated. 2. **Reports & disputes queue** — a place for the "someone flagged this" workflow that any marketplace needs; suspending straight from a report closes the loop between "a problem was reported" and "an admin acted on it." 3. **Activity log / audit trail** — accountability for who did what, which also makes the console testable/demoable (you can see the effect of your own actions). 4. **Admin team/roles list** — a console with a hardcoded single admin user doesn't reflect that real platforms have multiple staff with different permission levels. 5. **Reset Demo Data** — a prototype-specific affordance so the demo doesn't get "stuck" in a suspended/modified state after someone tries the buttons. 6. **Admin login gate** (`admin/login.html`) — the original ask assumed an admin section existed on the site already reachable somehow; since it didn't, this is the entry point, reachable from `signin.html`. 7. **Full customer/vendor detail pages** (`customer-detail.html`, `vendor-detail.html`) — requested directly: monitoring "all their account activity and user data" needed more room than a quick-look popup could give, so each account now gets its own page with full profile info, a filtered activity history, and any reports against it. 8. **Add/Remove Admin** on Settings — also requested directly: the console needed a real way to grow (or shrink) its own staff list, not just display three hardcoded names. 9. **Password reset for customers and vendors** — requested directly: an admin console that can suspend an account but can't help someone locked out of it is missing a basic support tool. 10. **Admin profile photos** — requested directly, with a real (client-side) upload rather than a placeholder, since the console already has somewhere to persist it. 11. **Per-admin activity attribution + role-gated activity browsing** — requested directly ("let super admins see the activities of other admins... let all admins see which admin attended to a case"): every log entry and every report now records who acted on it, visible to all; but *browsing* the full log is Super-Admin-only, with everyone else scoped to platform events and their own actions. 12. **Admin invite + simulated email verification** — requested directly: adding a new admin now goes through an invite-and-verify step instead of adding them outright, standing in for the real "confirm your email" step a production onboarding flow would have. 13. **Guest checkout, on by default** — requested directly: `admin/settings.html`'s "Allow guest checkout" toggle now starts checked, and `signin.html` gained a **"Continue as Guest →"** link (buyer tab only) straight into `customer/dashboard.html`, so the platform-control setting has a matching entry point on the actual sign-in page. 14. **A vendor-verification kill switch** — requested directly: `admin/settings.html`'s Platform Controls gained a "Require ID/business verification for new vendors" toggle (on by default), alongside the pre-existing "Require approval for new vendors" one — the two are deliberately separate switches, since account approval and KYC review aren't forced to move together (see §7's vendor-detail.html note). 15. **Order visibility for both sides of a transaction** — requested directly ("let admins be able to check ordered and sent out items from the customer and the vendor"): before this, a customer/vendor detail page only showed lifetime order *counts*, with no way to see an actual order, its item, or its shipment status. Added a shared `orders` sample dataset to `admin/assets/data.js` (`getOrdersForCustomer()` / `getOrdersForVendor()`) and a **Recent Orders** list to both detail pages — see the updated descriptions above. 16. **A real cart, real product-detail pages with reviews, and working category filtering** — requested directly, reporting that Add to Cart didn't actually add anything, cart.html showed hardcoded dummy items, clicking a product showed nothing, and clicking a category didn't filter. All four were

real: there was no shared product catalog anywhere in the customer app, so a product card had nowhere to link to and nothing to persist into a cart. Built `customer/assets/products.js` (a shared 23-product catalog — the same "Gaming Laptop" the dashboard and Godsfavour Laptops' store page both show is now one catalog entry, not two coincidentally-matching listings) and `customer/assets/cart-store.js` (`CartStore`, `localStorage`-backed); rewrote `cart.html` / `cart.js` to render real cart contents with no dummy data; added `customer/product.html` (detail + reviews, same pattern as `store.html`'s); wired every product card across dashboard/explore/category/store to both `CartStore` and click-through; and gave `category.html`'s category tiles (and `dashboard.html`'s category rail) a real `filterByCategory()` instead of a dead `href="#"`. Found and fixed one bug of its own along the way: `product.html`'s Add to Cart button initially reused the `.add-cart` class the card-level delegated handler also matches, which — since that button has no `[data-product-id]` ancestor card — made the *global* handler's fallback fire and navigate the whole page to `cart.html` on click, on top of the page's own handler. See §5's updated page descriptions and interaction reference above. 17. **Merged Category into Explore, added Orders to the nav, moved Support into Profile** — requested directly, as an information-architecture cleanup: Category and Explore were two separate pages doing almost the same job (identical filter-panel + product-grid pattern; Category's only real addition was its 8-tile "shop by category" grid), Order History had no nav entry at all (only reachable from Settings and one notification card, despite being something every buyer needs often), and Support — a rare action — was taking up a permanent nav slot on both the sidebar and the bottom tab bar. Moved `category.html`'s browsing job, its 2 extra filter-panel category chips, and its 6 products into `explore.html` (now 12 products total); turned `category.html` into a `window.location.replace('explore.html' + location.search)` redirect stub so old `category.html?cat=X` links keep working; updated every internal link that pointed at `category.html` (`dashboard.html`'s category rail, and every page's own nav) to point at `explore.html` directly. Removed the `.support-btn` sidebar link and bottom-nav item from all 9 pages that had it and added a "Contact support" button to `settings.html`'s new **Help & Support** card instead — no JS changes needed there since `support.js`'s click listener is already delegated on `document`, so it fires for a `.support-btn` anywhere on the page. Added a new **Orders** entry (originally a package icon, changed to a receipt icon in item 20 to match the vendor app's own Orders icon) to both the sidebar and bottom-nav on all 9 customer pages, in the slot Category vacated, marked active on `orders.html` itself. Net nav change: bottom-nav went from 5 items (Home, Category, Explore, Support, Profile) to 4 (Home, Explore, Orders, Profile); sidebar went from 6 (Home, Category, Explore, Chat, Support, Profile) to 5 (Home, Explore, Orders, Chat, Profile). **Follow-up, same session:** the initial merge carried `category.html`'s 8-tile image grid over as-is; direct feedback said to drop the grid entirely and represent categories as plain filter buttons instead, matching the trending-chips row already on the page. Replaced `.categories-grid` / `.category-card` with 8 `.category-chip` buttons styled via the existing `.trending-chip` class (added `font-family: inherit; cursor: pointer;` to that class so it renders correctly on a `<button>`, not just an `<a>`), and expanded the filter panel's category checkboxes from 5 to 9 (every chip's category, plus Sports) so every quick-filter chip now has a real matching checkbox — closing the gap where a clicked category without one used to need a special-cased "no products yet" empty state; now every chip resolves through the same `applyFilters()` path a manually-checked box would. Verified live across all 9 pages: zero console errors, consistent nav item counts/labels, chip-based filtering and the `?cat=` deep link both work, the `category.html` redirect preserves its query string, and the settings.html support button still opens live chat (`body.livechat-open`). 18. **Real "New"/"Hot" product badges instead of 5 hardcoded ones** — asked directly to either

remove the Hot/Trending/New badges or wire them up to real logic and document the real-backend version, choosing the latter to stay consistent with how every other "fake claim" gap in this prototype (cart, reviews, KYC docs, orders) has been handled this session — build it real, don't delete the feature. The 5 existing badges (`explore.html` only — the identical products on `dashboard.html` showed none) were plain hardcoded `<span>`s with no data behind them, and "Trending" turned out to just be alternate text on the same `.hot` CSS class, not a real third category. Added `createdAt` / `salesCount` to every entry in `products.js`'s catalog and a new `getProductBadge(product)` function there (New = created within 21 days, else Hot = `salesCount >= 50`, else no badge — New wins if both apply); added `assets/badges.js`'s `decorateProductBadges()` to apply that to the DOM on `dashboard.html`, `explore.html`, and `store.html`; removed the 5 hardcoded spans from `explore.html`. Documented the real-backend version in `BACKEND_GUIDE.md` §3 ("Product badges") and its `GET /api/products` / `GET /api/products/:id` response shape in §5 — `created_at` is an ordinary row timestamp, `sales_count` is a `COUNT` / `SUM` aggregate over completed `order_items`, and the front-end's threshold logic doesn't need to change once those two fields come from real columns instead of hard-coded catalog values. See §5's updated page descriptions above. 19. **Security card (`settings.html`) hidden until Edit, and dropped the fake "Current password" field** — requested directly. Previously the whole password form was always visible but `readOnly` / `disabled` until "Edit" was pressed (a locked-but-shown state); it now follows the exact same show/hide pattern the Profile card above it already used — `#password-form` carries `.hidden` by default and Edit/Cancel/submit toggle that class, rather than toggling `readOnly` on inputs that stay on-screen the whole time. Also removed the `#current-password` field entirely: there's no backend to check it against, so asking for it was asking the user to type a real secret into a field that verified nothing — the form now only asks for the new password twice, honest about what this prototype can actually do (nothing, yet — see the existing `// TODO: wire up to real update-password endpoint`). Cleaned up the now-dead `.field-group input[readonly]` and `#password-submit-btn:disabled` CSS rules in `settings.html`'s own `<style>` block, since neither state exists anymore. See the updated `settings.html` interaction reference above. 20. **A real "See more" destination for Top Vendors, plus two nav follow-ups requested in the same breath** — three related asks: (1) `explore.html`'s "TOP VENDORS" → "See more" was an inert `<button>`; it should go to a vendors page with a name search. (2) Change the Orders nav icon. (3) Add Chat/Messages to the mobile bottom nav (it was sidebar-only). For (1): extracted `store.html`'s previously page-local `VENDORS` object into a new shared `assets/vendors.js` (`getVendor()` / `getAllVendors()`, same pattern as `products.js`) so the data isn't duplicated a third time on top of the two-vendor-lists gap §8 already documents; built `customer/vendors.html` — a directory grid of every vendor (avatar, name, `verified` / `pending` status pill, category, rating, location) with a live `#vendorSearchInput` that filters by name and an honest "No vendors match" empty state, each card linking to `store.html?vendor=<id>`; pointed the "See more" `<a>` at it. For (2): swapped the Orders nav icon (a package/box, from item 17) for the receipt icon `vendor/dashboard.html`'s own Orders nav link already uses, for visual consistency between the two apps' Orders entries — same 5 icon instances per page (sidebar) plus 5 more (bottom-nav) across all 10 customer pages now share this one icon. For (3): added a Chat bottom-nav item (icon + label, matching the sidebar's) to all 10 customer pages, marked active on `chat.html` — the sidebar and bottom nav now list the identical 5 items (Home, Explore, Orders, Chat, Profile) instead of the bottom nav dropping Chat as it did right after item 17. Verified live: `explore.html`'s See more → `vendors.html` → search "jen" narrows to 1 card → click-through lands on the correct `store.html?vendor=` page with zero console errors; both navs render 5 matching

items site-wide. 21. **Picture-only dashboard banner, made admin-editable** — two related asks in the same breath: (1) the dashboard banner's promo text and color block were covering half of every image — make it picture only. (2) Move the banner images into their own file, and give admins a way to change the site-wide banner from the console. For (1): removed the `.banner-text` block from both of `dashboard.html`'s slides entirely (not just visually hidden) and made the `<img>` fill the slide at 100%×100%; this took a second pass to actually land — the first attempt left a visible sliver of the `.banner` gradient around the image because of a pre-existing, undocumented second `.banner-slide { padding: 20px }` rule later in `style.css` that this project's own CSS comment already flagged as needing reconciling — fixed by scoping every picture-only override under one new `.banner--picture-only` class (2-3 class specificity) rather than fighting the cascade — `explore.html`'s text+image banner was deliberately left alone in this pass (see item 22 for why that changed right after). For (2): moved the banner image files from `customer/assets/images/` into a new shared `assets/banners/` at the project root (alongside the existing `assets/brand/` convention every app already references via `../`); added a `siteBanners` array to `admin/assets/data.js`'s state (`getSiteBanners()` / `addSiteBanner()` / `removeSiteBanner()` / `moveSiteBanner()`, seeded with the 2 moved images) and a new **Site Banners** card to `admin/settings.html` (thumbnail + Up/Down reorder + Remove per row, plus an "+ Add banner image" file picker using the same `FileReader`-to-base64 pattern as the admin avatar upload — no real file storage exists yet, same caveat as that feature). `dashboard.html` now loads `../admin/assets/data.js` directly and builds its banner slides from `VetraAdmin.getSiteBanners()` on load (falling back to 2 default images if unavailable) — this is the **second** place (after `report-issue.js`) a customer page reaches into admin's `localStorage` state on purpose. Verified live end-to-end: removing a banner in admin Settings and reloading `dashboard.html` shows one fewer slide, with zero console errors throughout. See `BACKEND_GUIDE.md`'s new `site_banners` table (§3) and `/api/site-banners` routes (§5) for how this should work once a backend exists — including replacing the direct `localStorage` read with a real `fetch()` call, the one part of this feature that doesn't fit the rest of the site's eventual API-based shape. 22. **The same picture-only, admin-managed banner extended to** `explore.html` — asked directly ("do the same thing for the explore section") right after item 21 shipped for `dashboard.html` only. `explore.html`'s banner previously had its own separate hard-coded text+image slides ("Explore" / "Featured Vendors" copy, different from `dashboard.html`'s), which would have silently drifted out of sync with whatever an admin sets in Site Banners otherwise. Applied the exact same treatment: emptied the slide markup down to `.banner.banner--picture-only` + `.dots` shells, loaded `../admin/assets/data.js`, and copied the identical banner-building inline script from `dashboard.html` (same `DEFAULT_BANNERS` fallback, same `Vetra.wireBannerCarousel()` re-wiring after building slides). Updated `admin/settings.html`'s Site Banners description and `admin/assets/data.js`'s `siteBanners` comment, both of which previously said "the buyer dashboard" singular, to name both pages — this list is genuinely site-wide now, matching what its own label already claimed. Verified live: removing a banner in admin Settings now removes it from both `dashboard.html` and `explore.html` on reload, with zero console errors on either page. 23. **Capped the vendor Orders list to 5 rows with a "Show more" button** — requested directly. `vendor/orders.html`'s "ALL ORDERS" list (9 static rows) previously rendered every row at once regardless of the active status filter. Rewrote `vendor/assets/orders.js`'s `wireOrderFilterTabs()` around a single `applyVisibility()` function: it first narrows to rows matching the active tab, then shows only the first `ORDERS_MAX_VISIBLE = 5` of *that* filtered set (not 5 of the unfiltered list, which would've hidden rows a filter should surely include), hiding the rest and showing a **"Show N more orders"** button (`#orders-show-more-`

`btn`, new — styled to match the page's existing button language) whose label reflects exactly how many are hidden and which disappears entirely once nothing's left. Clicking it reveals the rest for the current filter; switching filter tabs resets back to a capped view rather than staying expanded, so paging never leaves a stale "everything's already shown" state behind after a filter change. Verified live: loads with 5 of 9 visible and a "Show 4 more orders" button, Show more reveals all 9 and hides the button, filtering to "Processing" (2 matches) shows both with the button correctly hidden, and returning to "All" re-caps to 5 — zero console errors throughout. 24. **Full codebase audit for errors and unused code, then a cleanup pass on every finding** — requested directly. Ran `node --check` on every `.js` file site-wide, live-loaded every page across all four apps (plus `backend/`) checking for console errors and broken `<script>` / `<link>` / `<img>` references, and scanned every stylesheet for classes with zero textual match anywhere in their app (including inside JS template strings, not just static `class="..."` attributes) — this last check was done twice: once per-app by background agents, once again by hand with a small Python script (parses each stylesheet into top-level rule blocks, flags any whose selector's classes are **all** absent from that app's HTML/JS) so the removal step below could be automated rather than hand-verifying dozens of rules one at a time. Zero functional bugs were found anywhere — no syntax errors, no broken links, no broken `id` references, no live console errors. Everything found was dead weight: - **Dead functions removed**: `customer/assets/products.js`'s `getAllProducts()`, `customer/assets/vendors.js`'s `getVendor(id)` (both `store.html` / `vendors.html` resolve products/vendors a different way already), and `admin/assets/data.js`'s `isSuperAdmin()`, `getActivity()`, `getOrders()` (each had zero call sites, including from within `data.js` itself — `getVisibleActivity()` / `getOrdersForCustomer()` / `getOrdersForVendor()` are what every page actually calls). - **Dead CSS removed**: ~102 fully-unused rule blocks from `vendor/assets/style.css` and ~27 from `admin/assets/style.css` — both files had a large chunk of customer-storefront-shaped CSS (banner carousel, chat widget, categories rail, cart, generic `.product-card`) copy-pasted in that neither app's own pages ever use; a few more compound selectors (`.chat-tab.active`, `.stock-pill.low`, etc.) were removed by hand since their base class was dead even though the modifier half (`active` / `low`) is a generic class used elsewhere for other elements. One 3-way grouped selector in vendor's stylesheet (`.page-card, .summary-card, .notification-card { ... }`) was trimmed rather than removed outright — `.notification-card` is genuinely live in `vendor/notifications.html`, so only the two dead names were dropped from the group, its styling left intact. - **The `customer/assets/style.css` duplicate/conflicting `.banner` block flagged in items 17/21 was finally reconciled**, not just worked around: since both `dashboard.html` and `explore.html` are picture-only now (item 22), the old text+image variant (`.banner-text` + 4 sub-rules, `.banner-art`) was 100% dead and removed outright, and the two competing copies of `.banner` / `.banner-slide` / `.dots` (previously ~140 lines total, including a second pair of conflicting `height` values inside two separate `@media (min-width: 900px)` blocks) were merged into one definition — the temporary `.banner--picture-only` modifier class from item 21 no longer needed to exist as a separate scope and was folded directly into the base `.banner` / `.banner-slide` rules, so `dashboard.html` / `explore.html` now just use plain `class="banner"`. One judgment call: the two conflicting copies of `.dots span.active`'s color (`var(--navy)` vs. a hardcoded `#7b5cf0` duplicate of `--purple1`) couldn't both survive — kept the token-based one as the more clearly-intentional version rather than the hardcoded copy-paste duplicate; the visual difference is a single small carousel dot and was a pre-existing inconsistency either way, not something this pass introduced. - **Root-level `style.css` dead code removed**: `.eyebrow.on-dark`, `.logo .dot`, `.hero-proof` (+ `b`),

`.stats` / `.stat` (+ `b` / `span`, plus a responsive override), `.timeline` / `.tl-row` (+ `:last-child` / `h4` / `p`)/ `.tl-year` (plus a responsive override) — roughly 90 lines from an earlier design (a stat bar, a timeline component) that no longer exists on any page. `main.js` — loaded only on `signup.html`, where both of its behaviors (nav-link highlighting, a mobile menu toggle) targeted elements that don't exist on that page, making it fully inert — had its `<script>` include removed from `signup.html` and, since nothing else referenced it, the file itself was deleted. - **Orphaned images removed**: `customer/assets/images/cat-electronics.jpg`, `cat-sports.jpg`, `cat-supermarket.jpg` — leftover from the "shop by category" tile grid that item 20's follow-up replaced with plain text buttons. (A fourth candidate, `cat-home.jpg`, was *not* removed — it looked unused from inside `customer/` alone, but a whole-repo check found the root `index.html` still references it directly.)

```
Verified after every removal: `node --check` re-run on every touched `.js` file, every touched stylesheet re-scanned with the same dead-selector script (zero remaining), brace-balance checked on both edited stylesheets, and 12 representative pages spanning all four apps live-loaded with zero console errors.
```

25. **Reviews only leavable after an actual purchase** — requested directly. Both review forms (`store.html`'s, per-vendor; `product.html`'s, per-product) previously showed a hint claiming this was already enforced ("this form simulates that check") while accepting a submission from literally anyone — there was no data anywhere connecting a browser session to a completed order. Built `customer/assets/purchase-history.js`'s `PurchaseHistory`: a `localStorage`-backed list of purchased product ids (`vetra_customer_purchases`), with `recordPurchase(ids)`, `hasPurchased(productId)`, and `hasPurchasedFromVendor(vendorId)` (true if *any* recorded product's `vendorId`, looked up via `products.js`'s `getProduct()`, matches). Wired `recordPurchase()` into `cart.js`'s `wireCheckoutButton()`, right before `CartStore.clear()` — so a real "Add to Cart" → "Checkout" round-trip through the site's own real cart is what unlocks the review form, not a separate/parallel mock. Both review forms now ship their `<form id="review-form">` with the `hidden` attribute and only reveal it if the relevant `PurchaseHistory` check passes on page load, swapping `#review-form-hint`'s text between a locked and an unlocked message; nothing was previously gating these beyond a sentence that admitted it wasn't real. Verified live end-to-end: a fresh (cleared `localStorage`) visit locks both forms; buying `oraimo-power-bank` unlocks `product.html?id=oraimo-power-bank`'s form specifically while a *different* unpurchased product (`smart-led-tv`) stays locked; buying `gaming-laptop` (`vendorId: gods-favour`) unlocks `store.html?vendor=gods-favour`'s form while a different vendor (`daniel-ice-fish`) stays locked — zero console errors throughout.
26. **Chat hidden site-wide, "for now"** — requested directly, explicitly as a temporary hide rather than a removal. Stripped the "Chat" sidebar-link and bottom-nav item from all 10 customer pages (back to 4 items each, matching the pre-item-20 symmetric count); removed the 5 "Contact vendor" buttons from `orders.html`'s order-track panels (leaving each card's real "Report an issue" button in place); removed `store.html`'s "Message Vendor" button and the inline script line that set its `href` programmatically. `customer/chat.html` itself — markup, inline script, mock conversation data — was **not** touched or deleted; it's simply unlinked from every other page now, still fully reachable (and working, including its `?chat=<id>`

deep link) if visited directly. No new file was created or removed for this change, only entry points into an existing one.

27. **Call button hidden on vendor storefront pages** — requested directly. `customer/store.html`'s `#store-call-btn` (a phone icon that only ever did `alert('Calling... — hook this up to your call flow')`) and its now-empty `.store-actions` wrapper were removed, along with the click handler that wired it. Store pages now show just the vendor header info and stats — no fake-call affordance pretending to be a working feature.
28. **A second, targeted audit pass — found and fixed a real regression from item 24's own cleanup, plus one pre-existing gap** — asked directly to "check for site wide degeneracies" after several rounds of edits. The most important finding: **item 24's dead-CSS removal broke `admin/dashboard.html`'s KPI icon colors.** `admin/assets/dashboard.js` builds each stat card as `class="stat-icon tint-${c.tint}"` (`tint` ∈ `blue` / `purple` / `green` / `red` / `amber`) — a JS template-literal the earlier cleanup's static-text scanner couldn't see, so it correctly found zero *literal* matches for `.stat-icon.tint-blue` etc. anywhere and removed them as "unused," even though they were live, just referenced dynamically. The bug was silent (no console error — `.stat-icon`'s base rule still applied, so every KPI card just quietly fell back to the same identical purple-blue tint instead of erroring), which is exactly why a targeted re-audit mattered: caught it by screenshotting `admin/dashboard.html` and noticing all 6 KPI icons were visually identical when they clearly shouldn't be. Restored all 5 `.stat-icon.tint-*` rules in `admin/assets/style.css`, reusing the same color tokens `.badge` / `.status-pill` variants already use elsewhere in the console (so "green" means the same thing everywhere, not a one-off). Re-audited every other JS file site-wide for the same failure mode (`class="...${var}"` template-literal construction) — found 10 more instances (`status-pill ${o.status}`, `badge ${r.status}`, `activity-dot ${...}`, `stock-pill${...}`, etc.) and checked each one's possible values against the CSS that's supposed to style them. All but one checked out fine (the literal status words happen to appear elsewhere as plain text, which is exactly why the scanner passed them correctly). The one exception: `.badge.rejected` **never existed at all**, in any point in this project's history (confirmed via `git log -p`) — a vendor's status can be set to `"rejected"` via `admin/vendors.js`'s reject action, and until now that badge silently fell back to unstyled/neutral instead of the red "this is a blocked account" treatment every other negative status gets. Added it, matching `.badge.suspended`'s colors. Also swept up leftover dead CSS this session's other edits created or missed: `.store-actions` / `.store-call-btn` (from item 27), `.delivery-options` and the whole `.categories-grid` / `.category-card` / `.category-hero` block (leftover from `dashboard.html`'s cat-card rail and the pre-item-20-follow-up category tile grid, missed by item 24's pass), and a `.chat-panel` class grouped into two `.chat-window` rules that no longer needs to exist (chat.html only ever used `.chat-window`). Verified: re-ran the same dead-selector script from item 24 against all four stylesheets, directly confirmed `.badge.rejected`'s computed colors match `.badge.suspended`'s, and live-loaded 25 pages spanning the public site + all three apps — zero console errors, all real content rendered. Correction (see item 30): that script only parsed top-level CSS rules, not ones nested inside `@media { ... }` blocks, so "zero remaining dead/mixed selectors anywhere" was not actually true — a real backlog of dead CSS buried in `@media` blocks (mostly vendor's leftover chat-feature rules) survived this pass undetected.
29. **Stat-icon color tints extended from admin to the vendor app, for visual consistency** — a design-polish follow-on from fixing item 28's regression, requested as part of "what UI updates can you make to enhance

the look." Vendor's dashboard/earnings/profile KPI cards never had a tint system at all (unlike admin's, which was *supposed* to but got broken) — every stat icon on `vendor/dashboard.html` (4 cards), `earnings.html` (4 cards), and `profile.html` (2 cards) showed the identical static purple-blue gradient regardless of what it represented. Added the same `.stat-icon.tint-blue/purple/green/red/amber` rules to `vendor/assets/style.css` (identical color tokens to admin's, so the two consoles' KPI grids now read the same color language) and assigned a semantic tint per card — green for revenue/balance, blue for orders/counts, amber for anything "pending," purple for inventory/brand-neutral metrics. Deliberately scoped to KPI-grid stat-cards only, not the visually-similar-but-differently-purposed per-row icons on `vendor/orders.html` and `earnings.html`'s Payout History list — those already carry a `.status-pill` for color-coding, so adding a second, redundant color signal there would be noise, not polish. Verified live on all three pages: correct tint per card, zero console errors.

30. **Vendor profile page redesign: real store background photo + a working profile-photo picker** — requested directly, reporting that the vendor's own profile page only had an avatar (no background/cover photo, despite customers seeing one on `store.html`) and that the avatar's "picture picker" button didn't actually let a vendor pick an image. Added a `.profile-cover` / `.profile-cover-wrap` banner section (defaults to a brand-gradient placeholder) above the avatar row, matching what `store.html` already shows buyers. Fixed the avatar button: it was firing `alert('Hook this up to your avatar upload flow.')` and nothing else — no `<input type="file">` even existed. Both photos now open a real device file picker and preview the chosen image immediately via an object URL, converting to a data URL on save and persisting it to `localStorage` (`vetra_vendor_profile_avatar` / `vetra_vendor_profile_cover`) so the change survives a reload, unlike a bare object-URL preview which dies with the page. See items 32/33 (which followed the same request) and item 36 (which fixed two follow-up problems with this exact feature) for how this evolved further in the same session.
31. **The "Edit Profile unlocks everything at once" pattern replaced with per-field inline editing, applied to all three apps** — requested directly for vendor's Store Details card, then extended to `customer/settings.html`'s Profile card and (since admin had no equivalent editable fields at all) a new Account Details card on `admin/settings.html`, to keep all three "edit your own profile" surfaces consistent. Previously, one "Edit"/"Edit Profile" button unlocked every field in the card simultaneously (vendor: `readonly` attributes cleared on all 6 inputs at once; customer: an entirely separate hidden form swapped in for a read-only `<dl>`) — clicking Edit to fix one typo put five other fields into an editable state too. Redesigned as: every field defaults to plain read-only text with its own small pencil button; clicking it swaps just that field into an editable input with a green check / red x pair, confirming or discarding only that field, everything else staying locked the whole time. Verified live (all three apps): clicking one field's pencil leaves every sibling field's view still showing as locked text, not an input.
32. **KYC panel actually closes after submission and reopens on rejection; admin gains submission date-and-time and a persisted rejection reason; admin roles documented for the backend build** — three related asks in the same breath. (1) `vendor/profile.html`'s KYC card previously never closed at all — submitting just disabled the fields while leaving the whole form fully visible and grayed out, which reset back to a blank "Not submitted" form on every reload regardless, since nothing was persisted. Rebuilt `vendor/assets/kyc.js` around a `{status, cacNumber, idFileName, cacFileName, submittedAt, rejectionReason}` object saved to `localStorage` and a real 4-state machine

(`not_submitted` / `pending` / `verified` / `rejected`) — see the KYC writeup in §6 above for exactly what each state shows. The `rejected` branch (form reopens, pre-filled, alongside the last submission's summary and the rejection reason) is fully implemented and correct but not reachable from this page alone in the current prototype, since only a real admin decision can set it and the two apps deliberately don't share state — documented for when a real backend closes that gap. (2) `admin/vendor-detail.html`'s KYC review only ever showed a submission *date*, not the time, despite being asked for "submission date and time" — added `VetraAdmin.formatDateTime()` (`admin/assets/data.js`) alongside the existing date-only `formatDate()`, used it for both the "Submitted" and "Reviewed" lines. Also found and fixed a related gap while implementing this: `VetraAdmin.setVendorKycStatus()` only ever wrote a rejection's reason into the activity log, never onto the vendor's own `kyc` record — meaning re-opening the vendor's page later showed *that* it was rejected but not *why*; now `kyc.rejectionReason` is set/cleared on the record itself and shown next to the "Reviewed" line. (3) Documented a full Super Admin / Moderator / Support permission matrix in `BACKEND_GUIDE.md` §4 (who should be able to suspend accounts, review KYC, resolve reports, manage the admin team, change platform settings), flagging that the real backend currently only enforces the Super-Admin-exclusive routes and nothing distinguishes Moderator from Support yet — a real gap, not a design choice. Also documented the full KYC data model, status lifecycle, and the real `GET` / `POST /api/vendors/me/kyc` + `PATCH /api/admin/vendors/:id/kyc` routes in `BACKEND_GUIDE.md` §3/§5, including exactly how the frontend's closed/reopened panel should get its status from a real backend instead of `localStorage`. Verified live: submit → panel closes with a persisted summary → reload keeps it closed → simulating a rejection (the only way to reach that state without a backend) reopens the form pre-filled with the rejection reason shown; admin's Reject flow now shows the reason next to "Reviewed `<date, time>`".

33. **Native browser `confirm()` / `alert()` popups replaced with an in-app modal, site-wide across vendor and customer** — requested directly ("I don't want it being the browser window that goes 'server says'... I just want a pop up that goes with the UI"). The admin console already had exactly this (`AdminUI.confirm()` / `.info()` in `admin/assets/ui.js`, see §7's "shared confirm modal") — every other native dialog site-wide was found and replaced with the same pattern, ported per-app as `VendorUI` / `CustomerUI` (`vendor/assets/ui.js`, `customer/assets/ui.js`) reusing the identical `.modal-overlay` / `.modal-panel` markup both apps' existing modals (Add Product, the report-an-issue modal) already established, so no new CSS pattern was needed. Replaced: vendor's sign-out/deactivate-store/delete-account/change-password/product-remove/product-edit/add-product-photo-required confirms and alerts; customer's deactivate-account/manage-payment/star-rating-validation (both `product.html` and `store.html` review forms)/cart's save-for-later and order-placed alerts/chat's call-fallback alert. At the time, each page that needed the modal included the shared markup block once, with `VendorUI` / `CustomerUI` falling back to a native `confirm()` / `alert()` if that markup was missing — item 40 later replaced this per-page copy with one self-mounting implementation shared across all three apps, so that fallback case no longer arises. Verified live across both apps: a `page.on('dialog', ...)` listener never fires for any of the replaced actions, confirming the native browser dialog genuinely never appears anymore.
34. **Vendor product categorization + a wider, more diverse site-wide category list, including Food** — requested directly ("let vendors viewing their already listed products be categorized... add more categories... add a category for food"). `vendor/products.html`'s product grid had no category information anywhere — no label on a card, no way to filter a large catalog by category. Added a category label to every

card plus filter tabs (`#product-filter-tabs`, `wireProductFilterTabs()` in `vendor/assets/product-actions.js`) above the grid, reusing the same `.filter-tab` / `.filter-tab.active` pattern `vendor/orders.html`'s status tabs already use; newly-added products (via the Add Product modal) also carry a category label/ `data-category` and respect whichever filter tab is currently active. Separately, the site's category taxonomy itself was inconsistent and missing Food entirely: the vendor Add Product dropdown had 8 categories (Appliances, Phones & Tablets, Health & Beauty, Home & Office, Electronics, Fashion, Computing, Gaming), `explore.html`'s filter-panel checkboxes had a different 9 (swapping Gaming for Supermarket + Sports), and its trending-chip row a different 8 still — none of the three matched, and none had Food, despite two seeded vendors (Daniel Ice Fish, Me n U Chops) selling nothing but food. Unified all three into one shared 13-category list (Electronics, Phones & Tablets, Computing, Gaming, Appliances, Home & Office, Fashion, Health & Beauty, **Food**, Sports, Books & Stationery, Baby & Kids, Automotive & Tools), applied identically to vendor's Add Product dropdown (both `dashboard.html` and `products.html`) and `explore.html`'s filter panel + trending chips. `customer/assets/filters.js`'s filtering logic is fully data-driven off whatever categories exist in the DOM, so no JS changes were needed there — a category with zero matching products on a given page still filters honestly to zero results (an existing, pre-built behavior) rather than doing nothing. Verified live: vendor's filter tabs correctly narrow the grid (5 Electronics / 2 Computing / reset to 9 on "All"), and `explore.html`'s new Food chip filters to zero with the correct empty-state message.

35. **Header avatar not updating on photo change, and the tiny save/cancel buttons redesigned as a popup — for vendor and customer (admin already worked)** — two related asks in the same breath. (1) Saving a new avatar on `vendor/profile.html` / `customer/settings.html` updated the photo on that page only — every page's header avatar icon (top-right) kept showing the old photo, even after navigating away and back, because nothing outside the profile/settings page's own script ever read the saved photo back out of `localStorage`. Admin already had this solved (`AdminUI.applyCurrentAdminAvatar()`, called on every admin page load) — ported the same idea to vendor (`VetraUI.applyCurrentVendorAvatar()`, `vendor/assets/interactions.js`) and customer (`Vetra.applyCurrentCustomerAvatar()`, `customer/assets/interactions.js`), both of which already load on every page in their app, plus called the same function immediately after a save so the header updates without needing a navigation first. Verified live: saving on the profile/settings page updates its own header instantly, and navigating to any other page in the app shows the new photo too. (2) The confirm/cancel controls for saving a new avatar/cover photo (added in item 31) were two 22px icon-only buttons crammed onto the corner of the photo itself — reported as too small. Replaced with `VendorUI.photoPreview()` / `CustomerUI.photoPreview()` (new methods on the item-34 shared-UI modules): a popup showing the picked photo large (circular for an avatar, wide for a cover banner) with full-size "Cancel"/"Save photo" buttons. Removed the now-dead inline check/x button markup and CSS (`.avatar-save-actions`, `.avatar-confirm-btn`, `.cover-save-actions`, etc.) from both `vendor/profile.html` and `customer/settings.html`. Verified live: picking a file opens the popup with the correct shape/title for avatar vs. cover, Save persists and closes it, Cancel discards the pick and leaves the previous photo (or the default gradient placeholder, for a cover that was never set) in place.

36. **A full codebase audit for broken UI/dead code, then every real finding fixed, plus `customer/chat.html` actually deleted this time** — requested directly, with the chat removal called out by name separately in the same session. The audit (four parallel passes, one per app area) found and this session fixed: an unescaped

`actorName` in `admin/assets/ui.js`'s `renderActivityFeed()` (a live join to `users.name`, a freeform field — any account name containing `<` / `>` / `&` broke the surrounding markup wherever that person's action appeared in an activity feed; this is what "the activity page UI gets broken" turned out to be); the Super Admin "all admins" filter `<select>` on `admin/activity.html` sizing itself to its widest `<option>` with `flex-shrink` disabled, overflowing its card on mobile's tighter width whenever a team member had a long name (the follow-up "vendor activity page" mobile report); `vendor/orders.html` / `earnings.html` missing `assets/ui.js` + the `#confirm-modal` markup entirely, so `VendorUI.confirm()` / `.info()` calls on those two pages threw and did nothing; contradictory password-reset copy between the list pages (falsely claiming "email delivery isn't configured, nothing was sent") and the detail pages (correctly claiming an email went out) for the exact same backend call; decorative notification/2FA toggles with no `id`/handler/backend column, now reverting and honestly saying "not wired up yet" instead of silently doing nothing; and `store.html`'s hand-rolled product-card markup (missing the qty stepper every other product grid has) switched to the shared `buildCustomerProductCard()` renderer, with its now-dead parallel CSS removed. Separately, `customer/chat.html` — unlinked since item 26 but never actually removed — was deleted outright this session (markup, inline script, mock data, and its CSS block in `customer/assets/style.css`), since nothing links to it and it was never migrated to the real backend like every other page was; see §5 and §9 point 10.

37. **Two-factor authentication wired up for real** — both places the UI already promised it (`admin/settings.html`: "Require a one-time code at admin onboarding," `vendor/profile.html`: "Add an extra layer of security at sign-in") were the decorative toggles item 37 above just fixed to at least stop lying — this closes the loop by actually building the feature they describe. `users.two_factor_enabled` (`backend/migrations/004_two_factor_auth.sql`), toggled via `PATCH /api/auth/me`'s `twoFactorEnabled`. When on, `/auth/signin` / `/auth/admin-signin` stop short of issuing a token and instead email a 6-digit code (reusing the existing Resend mailer and its console-log fallback) into a new `two_factor_codes` table — hashed, 10-minute expiry, 5-attempt lockout per code. `POST /auth/2fa/verify` completes the sign-in (this is also where `last_login_at`/the login activity-log row now get written when 2FA is on — not at the password step); `POST /auth/2fa/resend` issues a fresh code. `admin/login.html` and `signin.html` (vendor mode) both get a code-entry step that swaps in when the API responds with `twoFactorRequired`. See BACKEND_GUIDE.md's "Two-factor authentication" entry for the full endpoint reference.

38. **Vendor payout bank accounts verified against Paystack instead of trusting typed input** — requested directly ("use paystack miscellaneous api"). The payout form previously had a hardcoded 9-bank `<select>` and a freeform "Account Name" field the vendor just typed themselves, with nothing checking it matched what the bank actually had on file. The bank field is now a searchable `<input list>` backed by Paystack's Miscellaneous API (`GET /me/payout-account/banks`, cached server-side 24h); the account name is resolved live from the bank + NUBAN (`GET /me/payout-account/resolve`, debounced) the moment both are present, and `PUT /me/payout-account` **no longer accepts an** `accountName` **field at all** — it re-resolves server-side before writing, so what's stored is always the real bank-registered name regardless of what a request claims. Added `payout_bank_code` (`backend/migrations/005_payout_bank_code.sql`) so the Paystack bank code — not just its display name — is actually on file, which a future real transfer integration would need. See §6 and BACKEND_GUIDE.md's payout-account entries. **The** `<input list>` **described here was replaced by item 40 below** with a custom dropdown, for the reason given there.

39. **Modal/sidebar-chrome system consolidated into one shared file, plus a UI-polish and comment-rewrite pass.** `CustomerUI` / `VendorUI` / `AdminUI` each independently reimplemented `confirm()` / `info()` /the sidebar toggle, and the confirm modal's own `.modal-overlay` / `.modal-panel` markup was copy-pasted into 18 separate pages — exactly how `admin/activity.html` had once shipped without that markup at all and silently fell back to a native `window.confirm()` with nothing to catch it. Replaced with one project-root `shared-ui.js` exporting `VetraModal` (`confirm()` / `info()` / `photoPreview()`) and `VetraChrome` (`wireSidebarToggle()` / `wireSidebarCollapse()` / `applyAvatar()`); each app's own `assets/ui.js` is now a thin wrapper delegating to it, so every existing call site (`AdminUI.confirm(...)`, `VendorUI.info(...)`, `CustomerUI.photoPreview(...)`) keeps working unchanged. `VetraModal` builds its own markup and appends it to `document.body` the first time a page calls it, rather than depending on that page having pasted in a copy — see the corrected page descriptions in §5–§7 above, which previously (accurately, at the time) described the per-page markup requirement. The same commit replaced `vendor/earnings.html`'s native bank `<input list>` / `<datalist>` (unreliable on several mobile browsers, which don't render an `<input list>`'s suggestions at all) with a hand-built searchable dropdown — see §6's updated payout-account description — and added consistent card shadows, a richer sidebar gradient, tactile button hover/active states, and real empty-state icons across all three apps. It also rewrote comments site-wide (frontend and backend) to be shorter and more direct, and swapped unnecessary em dashes for commas in comments and in user-facing copy (error messages, emails, notifications, UI text) — a style pass with no behavior change, noted here for completeness since it touched effectively every file in the codebase.

40. **Cart-aware stock caps, verified-only Top Vendors, and a vendor-name overflow fix — all customer-app, one session.** The add-to-cart stepper on both the product grid (`customer/assets/product-grid.js`) and `product.html` only ever checked a product's total stock, never how much of it the shopper already had sitting in their own cart, so repeated adds across clicks or page visits could silently exceed real availability with nothing catching it until checkout rejected the order. Both now cap at stock minus whatever `CartStore.getQty()` reports is already in the cart, flipping to a disabled "Out of Stock" button the moment a click exhausts what's left — see the updated `product.html` description in §5 above. Separately, `explore.html`'s "Top Vendors" rail used to show the first 8 vendors regardless of KYC status; it now filters to `kyc_verified` vendors before slicing to 8, so an unverified store is never surfaced as a highlighted pick, and the vendor name is now escaped before being templated in (it wasn't before). `.vendor-mini-name`'s overflow/ellipsis rules had never actually engaged, since they sat on an unconstrained `inline-flex` element — split into a width-constrained row plus an inner truncating span so a long store name no longer overlaps the neighboring card. See §5's updated `explore.html` description above.

41. **Every avatar `<img>` now falls back to the placeholder on a failed load, and admin's customer/vendor lists show the real photo instead of always rendering initials.** A stale Google sign-up photo URL (or any other broken `avatar_url`) used to leave a broken-image icon showing in headers, profile pages, and `store.html`'s vendor avatar; every such `<img>` site-wide now carries `onerror="this.onerror=null;this.src='...avatar-placeholder.svg';"`. `customer/store.html`'s vendor avatar specifically used to start from an empty `src` with no fallback at all when a vendor was missing or unapproved; it now defaults

straight to the placeholder. Separately, `admin/customers.html` / `vendors.html` and their detail pages always rendered initials regardless of whether the account had a real `avatar_url` on file — `avatar_url` was added to the relevant `admin.routes.js` queries (`GET /customers`, `/customers/:id`, `/vendors`, `/vendors/:id`) and a new `VetraAdmin.avatarMarkup()` / `avatarInnerMarkup()` helper (`admin/assets/format-helpers.js`) renders the real photo with the same onerror-to-initials fallback, so those views now match what the customer/vendor apps themselves already showed. See §9 point 11 above, which this supersedes for the "only ever replaced once an account has a real avatar_url" half of that claim — it's now also true for the admin console's own lists, not just the customer/vendor/admin apps' own headers.

12. **Real client IPs behind Render's proxy, and a broken `login_ip_history` row layout, both fixed.** `app.set("trust proxy", 1)` only unwound one hop of Render's routing, so every row ever written to `login_ip_history` recorded Render's own internal `10.x.x.x` address instead of the real signer-in's IP — and every visitor looked identical to the rate limiters that key off the same `req.ip`. Replaced with Express's `"loopback, linklocal, uniquelocal"` preset, which walks past any number of private-network hops and returns the first real public address regardless of Render's exact topology; historical rows written before this fix are therefore not wrong data worth investigating, just useless. Separately, `admin/activity.html`'s `renderIpHistory()` had been reusing `.activity-item`, a 3-column grid built for an icon + message + time row, but only ever supplying 2 children — squeezing the IP address into the 34px icon column. Gave it its own two-column `.ip-history-row` layout instead. See `ADMIN_GUIDE.md`'s "Real client IPs" note and `BACKEND_GUIDE.md`'s `login_ip_history` entry for the fuller writeup.

13. **The one native `window.alert()` the item-34/item-40 modal sweeps had both missed, fixed.** `api-client.js`'s session-expiry handler (`redirectAfterSessionExpiry()`, fired from any authenticated `fetch` that comes back with an expired/invalid/revoked session) lived outside every per-app `assets/ui.js` file the earlier sweeps searched, since it's shared, project-root infrastructure loaded standalone on every app before any app-specific script — so it kept using a plain `window.alert()` long after every other native dialog site-wide had been replaced. Now calls `VetraModal.info()` (from `shared-ui.js`, see item 40) with the redirect moved into its `onClose` callback, so the "session expired" notice finally matches the rest of the app's own styling instead of the browser's default dialog chrome; falls back to the original `window.alert()` + immediate redirect only if `VetraModal` genuinely isn't defined (the two pages that don't load `shared-ui.js` at all: `admin/login.html`, `customer/category.html` — neither can hit this path in the first place, since neither ever holds a role token to expire).

14. **Hosting moved to cPanel.** The site is served from `vetrahq.org` and the API from `api.vetrahq.org` on one cPanel account, with its own MySQL and a cron job for the delivery and payments sweep. One Git deploy (`.cpanel.yml`) ships both. Render, Vercel and Clever Cloud are a short rollback window only. Email moved from Resend to the account's own SMTP mailbox.

15. **Delivery system and rider app.** VETRA riders, the rider app, dispatch, live tracking and escrow release; see `DELIVERY_SYSTEM.md`. Riders choose their own delivery area from the rider app.

16. **KYC through IDCheck.ng**, with BVN added (§9 point 28).

17. **Paystack payments, payouts and refunds** (§9 point 27, `PAYMENTS.md`): the cart's processing fee and "Pay securely" checkout, the payment-return handling on `customer/orders.html`, the rider app's Bank tab and payout statuses, the vendor Earnings page's commission and payout breakdown, and the Super Admin-only `admin/payments.html`.

— surfaced by a dedicated "deeply scan for where you could have hallucinated" audit, which caught that item 28's verification script only walked top-level CSS rules and never recursed into `@media { ... }` bodies, so a real backlog of dead selectors nested inside media queries went undetected and unremoved despite the "zero remaining" claim. Rewrote the scanner to recurse into `@media` / `@supports` blocks, then manually cross-checked every flagged selector against all HTML/JS in its app (including the two known scanner blind spots: template-literal-built classes like `.stat-icon.tint-*`, and files outside the app folder like root `password-toggle.js`) before removing anything. `vendor/assets/style.css` had by far the largest backlog — leftover chat-feature CSS from before chat was hidden (`.chat-input`, `.chat-shell` + children, `.chat-sidebar`, `.chat-header` / `.chat-header-left` / `.chat-back` — vendor never had a chat page at all, so all of it was dead on arrival), plus a `.categories-grid` / `.banner` / `.banner-section` / `.banner-text .off` / `.products-section` / `.cat-scroll` / `.cat-card` / `.cat-thumb` cluster left over from an earlier vendor storefront-preview layout nested inside the main `@media (min-width:900px)` block, and a genuinely-unused `.stat-icon.tint-red` (vendor's KPI cards use green/blue/purple/amber but never red — unlike admin, which uses all 5). `customer/assets/style.css` had the `.chat-panel` class named in item 28's own text as "no longer needs to exist" but never actually removed: a standalone `.chat-panel` rule, a standalone `.chat-sidebar` rule, and `.chat-panel` grouped into three compound selectors alongside the still-live `.chat-window` (trimmed to keep only the `.chat-window` half of each). Verified: brace-balance checked on both stylesheets, the `@media`-aware scanner re-run against all four stylesheets shows zero dead/mixed selectors anywhere except the two known-live template-literal false positives (admin's 5 `.stat-icon.tint-*` rules), and 9 representative pages across vendor and customer live-loaded with zero console errors and correct tint-icon colors confirmed via computed-style checks.

Full interaction reference (every button, exact behavior)

The page-by-page tour above says what each page is for; this is the granular version — every interactive control, the function that handles it, and where its state actually lives. All of it is driven through `VetraAdmin.*` (`admin/assets/data.js`) and `AdminUI.*` (`admin/assets/ui.js`), so those two files are named throughout rather than repeated per page.

Shared across every admin page (`admin/assets/ui.js`): - The header's sidebar-collapse icon (`#header-sidebar-toggle`) toggles `.hidden-desktop` on `#app-sidebar` and shows a floating reopen button (`#app-sidebar-reopen`) — `wireSidebarToggle()`. - The sidebar's own collapse arrow (`#sidebar-collapse-toggle`) toggles a `.collapsed` class (icon-only rail) — `wireSidebarCollapse()`. - Every header avatar image (`.header-avatar img`) is kept in sync with whichever admin is "logged in" — `applyCurrentAdminAvatar()`, called on every page load and again after a photo upload. - **The shared confirm modal** (`#confirm-modal`,

`AdminUI.confirm({title, bodyHtml, confirmLabel, danger, showReason, onConfirm})` is what every suspend/approve/reject/resolve/dismiss/remove action across the whole console routes through — no page wires its own modal. `danger: true` turns the confirm button red; `showReason: true` reveals an optional textarea whose value is passed to `onConfirm(reason)`. As of item 40, `AdminUI.confirm()` delegates to the project-root `shared-ui.js` (`VetraModal`), which builds `#confirm-modal` itself and appends it to `document.body` on first use rather than depending on a page's own static markup — this is what closed the gap where `admin/activity.html` had shipped without that markup at all and silently fell back to a native `window.confirm()`; that fallback path no longer applies, since there's no longer a markup-missing case to fall back from. - **The shared info modal** (`AdminUI.info({title, bodyHtml, confirmLabel})`) is the same overlay in a one-button mode, used only to show a value back to the admin (a generated temp password, "admin added" confirmation) — no `onConfirm`, no reason field. - **The shared activity-feed renderer** (`AdminUI.renderActivityFeed(container, entries)`) is what `dashboard.html`'s preview, `activity.html`'s full list, and both detail pages' "Account Activity"/"Activity History" sections all call — one function, so a change to how an entry renders (its icon, its "by X" attribution) updates everywhere at once.

`dashboard.html` (`admin/assets/dashboard.js`): - Six KPI cards are pure display, computed once from `VetraAdmin.getStats()` on load (`renderStats()`) — no interaction. - The "Pending Vendor Applications" table (`renderPendingVendors()`, capped at 4 rows) reads `VetraAdmin.getVendors().filter(v => v.status === "pending")`. Its **Approve** button opens a confirm dialog ("Their store and listings will go live immediately") whose `onConfirm` calls `VetraAdmin.setVendorStatus(id, "active")`, then re-renders the pending table, stats, and activity feed. **Reject** opens a `danger: true, showReason: true` confirm ("They can re-apply later") calling `setVendorStatus(id, "rejected", reason)`. - The Recent Activity preview shows `VetraAdmin.getVisibleActivity().slice(0, 6)` — role-scoped (see the Activity Log entry below), not the raw log.

`customers.html` (`admin/assets/customers.js`): - The search input (`#customer-search`, `input` event) and status dropdown (`#customer-status-filter`, `change` event) both call one `render()` that filters `VetraAdmin.getCustomers()` by `name` / `email` substring match (lowercased) and by exact `status`, then rebuilds the table body from scratch every time — there's no diffing, the whole `<tbody>` is replaced. - The status filter's initial value can come from the URL: `customers.html?filter=suspended` (used by `dashboard.html`'s "View Suspended Accounts" quick action) pre-selects that filter on load. - Each row's name/avatar cell and its **View** button both link to `customer-detail.html?id=<id>` — same destination, two click targets. - **Reset Password** opens a confirm dialog; on confirm, `VetraAdmin.resetCustomerPassword(id)` generates a temporary password (returned as a plain string — see `admin/assets/data.js`) and `AdminUI.info()` displays it in a `.reveal-panel` for the admin to copy and relay manually. This is the prototype's stand-in for "email a reset link" — see DOCUMENTATION.md §9 for why a real backend must never show this value to the admin. - **Suspend** (`danger: true, showReason: true`) calls `VetraAdmin.setCustomerStatus(id, "suspended", reason)`; **Reactivate** (shown instead, once suspended) calls the same function with `"active"` and no reason.

`vendors.html` (`admin/assets/vendors.js`): - Same search-input pattern as customers, plus **filter tabs** (`#vendor-filter-tabs .filter-tab`, click-to-activate) instead of a dropdown: All / Active / Pending Approval / Suspended. `?filter=pending` in the URL pre-activates a tab the same way `customers.html`'s query param

does. - `render()` always excludes `status === "rejected"` vendors from every view, even "All" — a rejected application isn't shown anywhere in this list (it still exists in `VetraAdmin.getVendors()`, just filtered out here). - A **pending** row shows **Approve**/**Reject** instead of Suspend; both behave exactly as `dashboard.html`'s versions (same confirm copy, same `setVendorStatus()` calls) — this file and `dashboard.js` each have their own copy of that logic rather than sharing one function, which is a small, acknowledged duplication (same pattern noted for `customer/assets/interactions.js` vs `vendor/assets/interactions.js` in DOCUMENTATION.md §8). - **Reset Password** behaves identically to the customers page, via `VetraAdmin.resetVendorPassword(id)`.

`customer-detail.html?id=<id>` (`admin/assets/customer-detail.js`): - On load, reads `id` from the URL and calls `VetraAdmin.getCustomer(id)`; if it returns nothing (bad/missing id, or the customer was removed by a demo-data reset), the page swaps to a `#customer-not-found` empty state and never calls `render()`. - `render()` populates every field (name, email, signup method, phone, address, joined date, last login with a relative "time ago" via `VetraAdmin.formatDateWithRelative()`) directly by element id, then calls five sub-renderers: `renderStats()` (orders/spend/status tiles), `renderActions()` (Reset Password + Suspend-or-Reactivate, same confirm/action pattern as the list view, but note the button click handler here is attached with `wrap.querySelector('[data-action="suspend"], [data-action="activate"]')` — a single selector matching whichever one actually rendered, since only one of the two exists in the DOM at a time), `renderOrders()` (see below), `renderReports()` (see below), `renderActivity()`. - **Recent Orders** (`renderOrders()`): `VetraAdmin.getOrdersForCustomer(id)`, sorted newest-first by `placedAt`. Each row shows the item, the vendor name (looked up live via `VetraAdmin.getVendor()`, so it always reflects the vendor's current store name), the amount, a status pill (pending/processing/shipped/out-for-delivery/delivered/cancelled — "completed" is relabeled "delivered" here, matching what a customer/vendor sees on their own tracking pages), and a carrier/tracking line when either is set. Section is `hidden` entirely (not an empty-state message) when this customer has no sample orders — same convention as the Reports section. - **Reports filed against this account**: `VetraAdmin.getReportsForTarget("customer", id)`. If empty, the entire section is `hidden = true` rather than showing an empty state — a customer with no reports shows no reports section at all, not a "no reports" message. - **Activity History**: `VetraAdmin.getActivityForTarget("customer", id)` — every log entry for this specific account, not the platform-wide feed, rendered through the same `AdminUI.renderActivityFeed()` used everywhere else.

`vendor-detail.html?id=<id>` (`admin/assets/vendor-detail.js`): - Same not-found/render split as customer-detail. `renderActions()` branches three ways instead of two: `pending` → Approve/Reject, `suspended` → Reactivate, else → Suspend — matching `vendors.html`'s three-state list view. - **Business Verification (KYC)** (`renderKyc()`, added when the KYC feature was built): reads `vendor.kyc` (`{status, cacNumber, idDocumentName, idDocumentUrl, cacDocumentName, cacDocumentUrl, submittedAt, reviewedAt}`). `not_submitted` shows a plain "nothing to review" message with no actions. `pending` shows the CAC number and both documents via `docChip(fileName, url)` — when a `url` is present this renders an actual clickable thumbnail (`<a target="_blank">` wrapping a small `<img>`) that opens the full document in a new tab, not just the filename; `docChip()` falls back to a filename-only chip if `url` is missing, and to "Not uploaded" in muted text if there's no file at all. (An earlier version of this only ever showed the filename — an admin asked to "Verify Documents" had nothing to actually look at; fixed by generating two clearly-watermarked "DEMO / SAMPLE"

placeholder document images under `admin/assets/images/kyc-samples/` and wiring every seeded vendor's `kyc` object to point at them, since this prototype has no real uploaded-file storage to reference instead.) Below the documents, `pending` also shows **Verify Documents** (`VetraAdmin.setVendorKycStatus(id, "verified")`) and **Reject** (`danger: true, showReason: true`, → `setVendorKycStatus(id, "rejected", reason)`) buttons. `verified` / `rejected` show the same fields plus a "Reviewed \<date>" line and no action buttons — a decision, once made, isn't reversible from this screen. - **Recent Orders** (`renderOrders()`): mirrors the customer-detail version — `VetraAdmin.getOrdersForVendor(id)`, newest-first, same status-pill labeling — but each row shows the buyer's name (`VetraAdmin.getCustomer()`) instead of a vendor name, since here the vendor side of the order is already fixed (it's this page) and the customer side is what varies per row. - Reports/Activity sections work exactly like the customer-detail page, scoped to `("vendor", id)` instead of `("customer", id)`.

`reports.html` (`admin/assets/reports.js`): - Filter tabs (`#report-filter-tabs`, All/Open/Resolved/Dismissed) work the same click-to-activate pattern as the vendors page; `?filter=open` etc. in the URL pre-selects one. - Every report card shows **Mark Resolved** and **Dismiss** when `status === "open"`; a report against a customer or vendor (not a product) additionally shows **Suspend Account**. Nothing shows once a report is resolved/dismissed — a decided report has no further actions here. - **Mark Resolved** → confirm → `VetraAdmin.setReportStatus(id, "resolved")`. **Dismiss** → confirm → `setReportStatus(id, "dismissed")`. - **Suspend Account** is the one three-way action on this page: its confirm dialog is `danger: true, showReason: true` with the label "Suspend & resolve," and its `onConfirm` calls **both** `VetraAdmin.setVendorStatus()` / `setCustomerStatus()` (whichever `report.type` is) **and** `VetraAdmin.setReportStatus(id, "resolved")` in the same click — suspending the account and closing the report out in one confirmation, rather than requiring two separate actions for what's conceptually one decision.

`activity.html` (`admin/assets/activity.js`): - Filter tabs narrow by `type` (Account/Vendor/Report/Order/Login/All) — pure client-side array filter on whatever `VetraAdmin.getVisibleActivity()` already returned. - **The one page whose behavior literally differs by who's logged in**: if `VetraAdmin.getCurrentAdmin().role === "Super Admin"`, an admin-filter dropdown (`#activity-admin-filter-wrap`) appears, populated from `VetraAdmin.getTeam()`, letting a Super Admin narrow the feed to one person's actions (`entries.filter(a => a.actorId === adminFilter)`, layered on top of the type filter). Anyone else instead sees a static note (`#activity-scope-note`) explaining they're seeing platform events plus their own actions only — and their query never even reaches for other admins' actions, since `getVisibleActivity()` itself is already scoped (see `admin/assets/data.js` below).

`settings.html` (`admin/assets/settings.js`) — the most interaction-dense page: - **My Profile**: `renderMyProfile()` shows whoever `VetraAdmin.getCurrentAdmin()` currently resolves to. **Change Photo** (`wireAvatarUpload()`) is a real, working upload: clicking the button forwards the click to a hidden `<input type="file">`, then a `FileReader` reads the chosen image as a base64 data URL and calls `VetraAdmin.setTeamMemberAvatar(me.id, dataUrl)` — which persists it to that admin's `localStorage` record, re-renders the profile card and team list, and calls `AdminUI.applyCurrentAdminAvatar()` so the header avatar updates immediately without a reload. Unlike the vendor/customer avatar pickers, this one saves instantly with no preview-and-confirm popup — admin's console already persists every action straight through `VetraAdmin.*`, so there's no separate "pending" state to confirm. - **Account Details**

(`wireMyProfileFields()`, item 32): the same per-field pencil-edit pattern as vendor's Store Details/customer's Profile card, generically wired off each field's `data-field` attribute. Confirming a field calls `VetraAdmin.updateTeamMemberProfile(me.id, {name|email: value})`, then re-runs both `renderMyProfile()` and `renderTeam()` so the header, the profile card, and the team list all reflect the change together. - **Admin Team** (`renderTeam()`): each row's **Remove** button opens a `danger: true` confirm; on confirm, `VetraAdmin.removeTeamMember(id)` either succeeds (re-renders the list) or returns `{ok: false, error: "last-super-admin"}`, in which case an `AdminUI.info()` dialog explains the block instead of removing anyone — the UI never lets the console reach zero Super Admins. - **Add Admin** is a two-step flow, not one form: 1. `wireAddAdminModal()` — the **Add Admin** button opens `#add-admin-modal` (name/email/role fields). Submitting calls `VetraAdmin.inviteTeamMember({name, email, role})`, which creates a pending-invite record with a freshly generated verification code, closes the invite modal, re-renders the pending-invites list, and immediately opens the verify modal for the invite that was just created. 2. `openVerifyModal(inviteId)` / `wireVerifyInviteModal()` — the verify modal just asks for a code (any non-empty value is accepted, per how this was explicitly requested — see DOCUMENTATION.md §7). Submitting calls `VetraAdmin.verifyTeamInvite(inviteId, codeEntered)`; on success it closes the modal, re-renders both the team list and pending-invites list, and shows an `AdminUI.info()` "Admin added" confirmation naming the new member's role. - **Pending Invitations** (`renderPendingInvites()`, section hidden entirely when the list is empty): each row's **Verify** button re-opens the same verify modal as step 2 above; **Cancel** opens a `danger: true` confirm calling `VetraAdmin.cancelInvite(id)`. - **Reset Demo Data**: a `danger: true` confirm warning that "any changes you've made in this browser will be lost," calling `VetraAdmin.resetDemoData()` (wipes the `localStorage` key and re-seeds it) and then hard-navigating to `dashboard.html`. - **Sign Out**: a plain confirm, no state change beyond navigating to `../signin.html` — it doesn't actually clear `currentAdminId`, so reopening any admin page in the same browser still resolves to the same "logged in" admin (a known limitation of simulating a session with a single localStorage field instead of a real one — see DOCUMENTATION.md §9). - Platform-control toggles (require vendor approval, auto-flag listings, guest checkout, maintenance mode) have no JS behind them at all — flipping one changes nothing else on the page or in `localStorage`; they're purely cosmetic, matching the equivalent switches on `vendor/profile.html`.

`login.html` — no dedicated JS file; the sign-in logic is a `<script>` block inline in the page itself. Clicking **Sign in to console** validates both fields are non-empty (turns the field's border red if not, via inline `style.borderColor`), then calls `VetraAdmin.setCurrentAdminByEmail(emailField.value.trim())` — a case-insensitive lookup against `admin/assets/data.js`'s seeded team roster by email only, **no password comparison happens at all**. The page doesn't say this anywhere on-screen (its subtitle just reads "Restricted access. Sign in with your admin credentials to continue.") — the password field is real-looking but the value typed into it is never read. If the email doesn't match any team member, `setCurrentAdminByEmail()` returns `null` and the "current admin" simply doesn't change — the page still redirects to `dashboard.html` regardless, where everything renders based on whichever admin was last successfully resolved (possibly from a previous session), not a fresh "no one is logged in" state. There's a 250ms `setTimeout` before the redirect purely for a "Signing in…" button-state effect, not for anything asynchronous actually happening.

Where the state actually lives (`admin/assets/data.js`)

Everything above eventually bottoms out in one `const VetraAdmin = (() => {...})()` module, backed by a single `localStorage` key, `vetra_admin_state_v1`. A few implementation details worth knowing if you're extending this file:

- `load()` reads that key on first script execution; if it's missing/unparseable (private browsing, first visit, cleared storage), it deep-clones the `SEED` constant and persists that instead — so the app never renders with truly empty state. It also backfills fields that didn't exist in older saved states (`pendingInvites`, `currentAdminId`, and — added with the KYC feature — a default `kyc: {status: "not_submitted", ...}` on any vendor record missing one), so a browser with state saved from before a feature existed doesn't crash reading `undefined.status`.
- **Every mutating function follows the same shape**: look up the record, mutate the in-memory `state` object directly (not immutably — no copies), call the module-private `save()` (which just `JSON.stringify`s the whole state back into `localStorage`), then call `logActivity()` with a human-readable message built from the record's own fields (never a generic "record updated"). This is why every action across the console is independently auditable — the log entry is written from inside the same function that made the change, not bolted on by whichever page called it.
- `logActivity(type, message, target, opts)`: `target` is `{type, id}` for anything with a detail page to link back to (`null` for pure login/system events). The fourth parameter, `opts.systemEvent`, is a later addition (added alongside the buyer-report-filing feature) that skips attributing the entry to `getCurrentAdmin()` — needed because a report filed from a customer's browser has no admin actor at all, and without this flag the log would wrongly credit whichever admin last signed in on that machine.
- `getVisibleActivity()` is the one function everything role-sensitive routes through instead of the raw `state.activity` array: a Super Admin gets it unfiltered; anyone else gets it filtered to `actorId === null || actorId === getCurrentAdmin().id`. This is a **client-side filter only** — see DOCUMENTATION.md §9 for why that's not a real permission boundary, and `backend/src/routes/admin.routes.js`'s `GET /api/admin/activity` for the server-side version that actually enforces it.
- `addReport()` (added for the buyer-report-filing feature) is the one function in this file explicitly designed to be called from *outside* the admin console — see `customer/assets/report-issue.js`, which loads this file directly via `<script src="../admin/assets/data.js">` rather than treating it as admin-only.

---

8. Site-wide review: redundancies, fixes, and what still needs attention

An audit pass was run across the public site, customer app, and vendor app (the admin section was excluded from the audit since it was just built and independently verified separately — every stat card, table, filter tab, search box, and confirm-modal flow on all six admin pages was clicked through in an automated browser test and confirmed to work, including that suspending a customer persists after a page reload). Everything below that was flagged as an actual bug was fixed in this session and re-verified with another automated browser pass (checked live: cart quantity/price/subtotal math, "Contact vendor" navigation, the checkout flow, the chat

Send button, both settings buttons, the vendor product grid after the JS consolidation, and the signup→signin vendor-mode handoff — zero console errors).

Fixed

Where	Problem	Fix
`customer/cart.html`	The **Checkout**, **Save for later**, quantity **–/+** steppers, and **Contact vendor** buttons had no JS behind them at all — clicking any of them did nothing. This is the site's core purchase flow, so it was the highest-impact finding.	Added `customer/assets/cart.js`: quantity steppers now recompute that line's price and the whole order summary (subtotal + total) live; "Contact vendor" opens the chat app; "Checkout"/"Save for later" give real click feedback (a processing state, then a confirmation) in place of the mock payment/save API this prototype doesn't have.
`customer/chat.html`	The **Send** button (and pressing Enter) did nothing — messages could never actually be sent in the demo chat UI.	Wired `sendMessage()` into the chat-window render: it appends the message to that conversation's history, renders it into the thread, updates the sidebar preview/time, and clears the input.
`customer/settings.html`	**"Manage payment methods"** and **"Deactivate account"** had no `id` and no handler — every other button on that page was wired, these two were an apparent oversight.	Gave both an id and a handler matching the same confirm+alert "hook this up to your API" placeholder pattern already used by `vendor/assets/profile.js`'s danger-zone buttons, so the whole site's mock-action convention now applies consistently here too.
`signup.js`	After validating the form, the Continue button just sat on "Creating account…" forever — the signup flow had no redirect at all, unlike sign-in.	Added the same delayed-redirect pattern `signin.js` uses, sending the new account to `signin.html` (or `signin.html#Vendor` if they registered as a vendor).
`signin.js`	Didn't handle a `#Vendor` URL hash, even though `signup.js` already produced/expected that pattern for its own hash-based deep link — a latent inconsistency that the signup fix above would otherwise have silently tripped over (landing on the Buyer tab after registering as a Vendor).	Added the matching hash check so `signin.html#Vendor` now pre-selects the Vendor tab, closing the gap between the two auth pages.
`vendor/assets/dashboard.js` + `vendor/assets/products.js`	Both files contained byte-for-byte identical `wireVendorProductActions()` / `wireAddProductButton()` functions — accidental copy-paste, not the intentional shared-CSS convention used elsewhere, so a future edit to one copy could silently drift from the other.	Merged both into one `vendor/assets/product-actions.js`, updated `dashboard.html` and `products.html` to load it, deleted the two duplicates. Re-verified Edit/Remove and the Add Product modal still work on both pages.

Reviewed and left as-is (working as intended)

- **Sidebar/bottom-nav links** across every customer and vendor page resolve correctly and are consistent sibling-to-sibling.
- `vendor/assets/add-product.js` **modal** (shared by `dashboard.html` and `products.html`) opens/closes correctly via its close button, Cancel, backdrop click, and Escape.
- **Contact form, FAQ accordion** (`script.js`) on the public pages work.
- Several buttons intentionally show a "hook this up" placeholder rather than navigate (vendor Edit product, avatar edit, change password) — this is the established, deliberate pattern for "the backend doesn't exist yet" across the codebase, not a bug, so these were left alone. (At the time of this note they used a native `alert()`; item 34 later replaced that with the shared in-app modal — the placeholder pattern itself is unchanged, only which dialog shows it.)
- `customer/assets/interactions.js` vs `vendor/assets/interactions.js` and `signin.js` vs `signup.js`'s buyer/vendor toggle logic are near-duplicates of each other by the same intentional per-area convention described in §2/§6 — low-priority, cosmetic overlap, not touched.

Still open (needs a human decision, not a quick fix)

- `customer/cart.html`'s "Visit store" link (previously "Contact vendor") only appears for cart lines whose product has a `vendorId` in `products.js` — most of the dashboard/explore/category catalog doesn't (see the two-vendor-lists note below), so it won't show on most lines. A real vendor-linked catalog would remove this gap entirely rather than needing a per-item workaround.
- **Two separate, unreconciled fictional vendor lists.** `admin/assets/data.js`'s `SEED.vendors` (the vendors admin actually manages, `v1`–`v7`) and `customer/store.html`'s own `VENDORS` object (`gods-favour`, `daniel-ice-fish`, `jennet-jeans`, `me-n-u-chops`, used across `store.html` and now `orders.html`'s report-filing) were built independently and only coincidentally overlap once (`Naija Home Essentials` = `v3`). A report filed against one of the other three shows up correctly in `admin/reports.html`'s main queue (see §5/§8) but won't deep-link to a real vendor-detail page, since its `targetId` doesn't match any admin vendor record. Fixing this properly means picking one vendor list and having both apps reference it — bigger than a quick patch, so flagged here rather than partially addressed.
- Everything in §9 below (auth, database, uploads, cart/chat persistence) — these are architecture-level gaps, not bugs in the existing code.

Feature gaps closed by a follow-up audit

A later audit pass (live-rendering check across 15 priority pages at mobile and desktop widths, plus a static sweep for orphaned images and dead JS) found zero rendering bugs but three gaps between what the site *claims* and what it actually lets someone do — all three closed in this session:

1. **No order history or delivery tracking**, despite "real-time delivery tracking" being advertised on `index.html` and `buyer-protection.html`, and vendors being told to "log tracking details" on `vendor-protection.html`. Added `customer/orders.html` (order history + expandable tracking timeline) and an

Update shipment modal on `vendor/orders.html` (status/carrier/tracking-number, per order) — see §5 and §6 above.

2. **No customer-facing review UI**, despite "verified reviews only" being a repeated selling point. Added a Reviews section (summary + review list + write-a-review form) to `customer/store.html` — see §5 above.
3. **No vendor-facing view of reports filed against their store** — admin had a full moderation queue (`admin/reports.html`) but a vendor had no way to even see a report existed, let alone respond to it, despite `vendor-protection.html` walking vendors through exactly that flow. Added a read-only **Reports against your store** panel to `vendor/orders.html` — see §6 above.

The same audit also found 5 image files with zero references anywhere in the codebase (the original `logo.png` before the brand-asset generation pass, two unused mark-only logo variants, and two duplicate `avatar-dummy.jpg` files shadowed by an identically-purposed `.png`). Moved to `extras/` rather than deleted outright, in case they're wanted later.

A fourth gap of the same shape was reported directly by name afterward, not found by an audit pass:

4. **No vendor-facing KYC/business-verification UI**, despite `buyer-protection.html` stating "every Vendor passes ID and business verification before their first listing ever goes live." Added a **Business Verification (KYC)** section to `vendor/profile.html` (CAC number + ID/CAC certificate upload, `vendor/assets/kyc.js`) and a matching review card to `admin/vendor-detail.html` (`VetraAdmin.setVendorKycStatus()`) — see §6 and §7 above.

A follow-up audit, specifically hunting for this exact pattern (a claim in the site's copy with no matching feature anywhere), found two more — both reported and then closed in the same session:

5. **No way for a buyer to actually file a dispute/report against an order.** `buyer-protection.html` and `vendor-protection.html` both describe a detailed "buyer reports an issue → escrow stays held → vendor gets 48 hours to respond → admin rules" flow, and `admin/reports.html` exists to review exactly that — but `customer/orders.html`'s "Report an issue" button just opened generic live chat, and `admin/assets/data.js` had no `addReport()` function for anything to call at all. Added a **File a report** modal to `customer/orders.html` (`customer/assets/report-issue.js`) and a new `VetraAdmin.addReport()` — see §5 above for how this one deliberately reaches across into the admin console's `localStorage`, unlike every other customer/vendor feature.
6. **No vendor UI to set or view a payout bank account**, despite `vendor-protection.html` describing payouts running "straight to the bank account on file," and `vendor/earnings.html` itself captioning payout-history rows "Payout to bank account" as if one already existed — the field was missing from both the vendor form and the admin data model, not just missing a form. Added a **Payout Account** section to `vendor/earnings.html` (`vendor/assets/payout.js`) — see §6 above.

Both of those were fixed in the same session they were found. A seventh gap of this shape was reported directly by name afterward:

7. **Admin couldn't actually see a vendor's submitted KYC documents, only a filename** — the "Business Verification (KYC)" card added for gap #4 rendered `idDocumentName` / `cacDocumentName` as plain text, so **Verify Documents** asked an admin to confirm something they had no way to look at. Generated two

clearly-watermarked "DEMO / SAMPLE" placeholder document images (`admin/assets/images/kyc-samples/id-document-sample.jpg`, `cac-certificate-sample.jpg` — deliberately generic and obviously-fake rather than resembling any real identity document or government certificate, since this prototype has no real uploaded-file storage to reference instead) and wired every seeded vendor's `kyc` object to point at them; `docChip()` in `admin/assets/vendor-detail.js` now renders an actual clickable thumbnail that opens the full document in a new tab — see §7 above.

---

9. What was fake, and what's real now

Status update: this section is now almost entirely resolved. Everything below described the frontend's behavior when this document was first written, before a real backend existed. **A real backend now exists in `backend/` and is deployed and running** (see `BACKEND_GUIDE.md` for the build plan, data model, and API reference, and `backend/README.md` for how it's actually deployed) — `customer/`, `vendor/`, and `admin/` are now all fully wired to it. Each point below is marked with what's true today.

1. **Authentication — ✅ real.** `signin.js` / `signup.js` and `admin/login.html` all call the real `/api/auth/*` routes: real password hashing (bcrypt), real JWTs, and a real server-side check that a signed-in admin is actually an admin (`requireRole("admin")` on every `/api/admin/*` route — opening `admin/dashboard.html` directly with no valid token now redirects to `admin/login.html`, it doesn't render). "Continue with Google" is also real now (`POST /api/auth/google`) — see BACKEND_GUIDE.md's `/api/auth` section.
2. **Admin roles — ✅ enforced server-side, and reflected in the UI.** `requireAdminRole("Super Admin", "Moderator")` gates every moderation action (suspend/reactivate/approve/reject a customer or vendor, KYC review, resolving/dismissing a report) — a Support admin's token gets a real `403` calling these directly. Managing the admin team itself (`DELETE /team/:id`, `POST /invites`, `POST /invites/:id/verify`, `DELETE /invites/:id`) and platform settings (`/settings`, `/site-banners`) stay Super-Admin-only, also enforced server-side, including the last-Super-Admin removal guard. The console's buttons now match: `admin/assets/session.js`'s `canModerate()` / `isSuperAdmin()` gate every one of these renders, so a Support admin simply doesn't see a Suspend/Approve/Reject/Resolve/Remove-admin/Add-admin/Add-banner button — it used to render the same as for any other role and just 403 on click. Platform Controls toggles are the one exception that's shown-but-disabled rather than hidden, since viewing current settings is allowed for any admin role, only changing them isn't.
3. **Admin invite/verification — ✅ fully real.** `POST /api/admin/invites` generates a real code, hashes it before storing, and `POST /invites/:id/verify` checks the hash for real rather than accepting anything. The code is now actually emailed to the invitee (cPanel SMTP, `src/utils/mailer.js`) — until `RESEND_API_KEY` is configured it still logs to the Render server as a fallback, same as before, but the code itself and the sending logic are both real either way.
4. **Password reset — ✅ fully real.** An admin clicking "Reset Password" (`admin/customer-detail.html` / `vendor-detail.html`) generates a real single-use token, stores only its hash, and emails a real link to `reset-password.html?token=...` — the account holder sets their own new password there

(`POST /api/auth/reset-password`), and the admin never sees the resulting credential. No email provider configured yet just means the link is logged server-side instead of sent, same graceful-degradation as point 3.

5. **A real database — ✅ done.** Every customer/vendor/product/order/report list on every wired page comes from the real MySQL schema (`BACKEND_GUIDE.md` §3, on the cPanel account's own MySQL), not hard-coded JS or `localStorage`.
6. `admin/assets/data.js` **is gone entirely.** All 8 admin pages call the real `/api/admin/*` routes directly via `api-client.js`; the handful of pure-display helpers it used to also provide (`initials()`, `formatDate()`, `formatDateWithRelative()`) moved to `admin/assets/format-helpers.js`, same call shape, nothing else changed.
7. **Platform-control toggles** on `admin/settings.html` — ✅ **all five real now** (`platform_settings` table, `GET` / `PATCH /api/admin/settings`). Guest checkout gates `POST /api/orders`. Require vendor approval controls whether a new vendor signup starts `pending` or goes live as `active` immediately. Require vendor verification blocks approving a pending vendor unless their KYC is already verified. Auto-flag listings scans every new/edited product against a restricted-keyword list and auto-files a real report (visible in `admin/reports.html`'s queue, same shape a buyer's own report uses) if one matches — it doesn't block the listing, an admin still decides. Maintenance mode blocks new signups and checkout with a clear error; browsing and signin stay up.
8. **File uploads — ✅ real**, backed by Cloudinary rather than local disk (`POST /api/uploads`). Vendor product images/video, vendor's own avatar *and* cover photo, vendor's KYC documents, customer's avatar, and admin's avatar are all wired to it now — nothing site-wide still previews-only-locally.
9. **Notifications — ✅ real, both apps.** `customer/notifications.html` and `vendor/notifications.html` used to ship as three permanently hard-coded cards each with nothing behind them; both now render real rows from `GET /api/notifications`, with a real unread-count badge on the header bell icon everywhere. Real events generate a notification server-side: a new order (→ vendor), a shipment status change (→ buyer), a vendor application decision or suspension (→ vendor), a KYC decision (→ vendor), and an account suspend/reactivate (→ customer).
10. **Chat — the frontend stub is gone; the feature was never built.** `customer/chat.html` (UI-only, hardcoded mock conversations, never migrated to the real backend) was deleted entirely this session (item 37) rather than left as dead unlinked weight — nothing links to it anymore because there's nothing left to link to. Buyer↔vendor messaging itself remains genuinely unbuilt on the backend (needs polling, not WebSockets, on shared/free hosting) — that part of the status hasn't changed, only the "half-wired stub sitting around" part has. **Cart and checkout are real** — `customer/cart.html` calls a real, idempotent `POST /api/orders` (a client-generated key prevents a stalled-network retry from creating a duplicate order), not just a `localStorage`-backed mock.
11. **Every account's default profile picture used to be a real stock photo of a specific person** (`imgs/avatar-dummy.png`), shown for every customer/vendor/admin regardless of who they actually were until they uploaded their own — now a neutral placeholder icon, same across all three apps, only ever replaced once an account has a real `avatar_url`. (Item 42 later added an `onerror` fallback to that placeholder site-wide, for when a stored `avatar_url` itself fails to load, and extended "shows the real photo" to admin's own customer/vendor lists, which had kept rendering initials unconditionally.)

12. `vendor/dashboard.html`**'s stat cards and Recent Orders, and** `vendor/earnings.html`**'s stat cards and Payout History, used to be permanently fake** — not a loading flash, genuinely hard-coded numbers/rows with no `render*()` function behind them despite comments naming ones that didn't exist. Both now pull real data: dashboard stats/orders from `GET /orders/vendor` + `GET /products?vendor=`, earnings figures keyed off each order's real `escrow_status`, and Payout History listing real released/held orders (not a fabricated payout-batch ledger, since no such table exists — see BACKEND_GUIDE.md §3's note on `platform_settings` for the same "don't invent a shape the schema doesn't have" reasoning). `vendor/profile.html`'s KYC submission, payout account, and Verified Vendor badge are likewise all real now instead of `localStorage` mocks.
13. **Account lifecycle — ✅ all real now.** Vendor's Change Password button used to say "Not wired up yet"; it now opens a real modal calling `PATCH /api/auth/password`. Both apps' Deactivate/Deactivate Store buttons used to be an "info" popup with nothing behind it; they now call `PATCH /api/auth/deactivate` (sets `status = 'suspended'`, same reversible end-state an admin-initiated suspension already has) and sign the account out. Vendor's Delete Account button used to be the same kind of no-op popup; it now calls `POST /api/auth/delete-account`, which scrubs the account's personal data, delists every one of that vendor's products, and marks the row `status = 'deleted'` (a real e-commerce platform can't actually `DELETE` the row without breaking other buyers' order history that legitimately still references it).
14. `customer/cart.html`**'s "Save for later" button used to be a no-op alert.** It's real now — `SavedForLaterStore` (`customer/assets/cart-store.js`) is a second `localStorage`-backed item list built from the same factory `CartStore` uses (the cart itself has never been backend-real, so this stayed consistent with that rather than becoming the one part of checkout that talks to a database). The button moves every current cart line into it and clears the active cart; a "Saved for later" section appears on the same page whenever it's non-empty, with a real "Move to cart" per item.
15. **Every timestamp shown to a Nigeria-based browser was silently 1 hour off** (`admin/activity.html`'s "a few minutes ago" showing as "1h ago" is what surfaced it) — the database pool had `dateStrings: true`, so a `DATETIME` column came back as a timezone-less string a WAT (UTC+1) browser parsed as its own local time. Fixed at the root (`backend/src/db.js`) rather than patched per-display, so it also silently corrected every other timestamp site-wide, not just the activity feed.
16. **Admins can now be promoted/demoted between roles, and the activity log can be pruned** — both Super Admin only, both new. `PATCH /api/admin/team/:id` changes an existing admin's role (blocked only if it would leave zero Super Admins); `DELETE /api/admin/activity/:id` and `DELETE /api/admin/activity` remove one entry or the whole log. Neither deletion route writes a fresh entry documenting the pruning — asked for directly, on the reasoning that clearing the log is meant to actually clear it, not leave a new trace behind every time. Removing an admin who has activity-log or invite history no longer fails outright either — those rows survive with the actor reference cleared instead of blocking the delete.
17. **Two-factor authentication — ✅ real.** `users.two_factor_enabled`, toggled from `admin/settings.html` / `vendor/profile.html`'s Security card via `PATCH /api/auth/me`. When on, sign-in stops at an emailed 6-digit code (`POST /auth/2fa/verify` / `/2fa/resend`) before a session is issued — see item 38 and BACKEND_GUIDE.md.
18. **Vendor payout account — bank + account name are now Paystack-verified, not vendor-typed.** `GET /me/payout-account/banks` (searchable bank list) and `GET /me/payout-account/resolve` (live NUBAN →

registered-name lookup) back the form; `PUT /me/payout-account` re-resolves server-side and no longer accepts a client-supplied account name at all — see item 39 and §6 above.

19. **A "Livestock" category was added** to the fixed 14-value category list (now 15) — `customer/explore.html`'s filter checkboxes and chip row, and both vendor apps' Add Product `<select>`. Prompted by a real vendor listing ("Rams, Goats and Cows") that had nowhere to go but "Food" before this existed.
20. **Out-of-stock is now a real, enforced state — ✅ done.** `products.status`'s `'out_of_stock'` value existed in the schema from day one but nothing ever set it; a depleted product just kept showing `stock_quantity: 0` while still being fully browsable and addable to cart. Now: checkout's stock-decrement and a vendor manually zeroing `stockQuantity` both flip `status` to `'out_of_stock'` (and back to `'active'` on restock), and every customer-facing route (`GET /products`, `/products/:id`, search) filters on `status = 'active'`, so a depleted listing actually disappears from browse/search/category pages and the product detail page instead of just failing at checkout. The Add to Cart button now disables itself at 0 stock (product grid, product detail page) instead of only capping the quantity stepper; the cart page flags a line that went out of stock after being added and blocks the checkout click with a clear message rather than only finding out from the backend's rejection. On `vendor/products.html`, a vendor's own out-of-stock listings moved into their own "Out of Stock" section (`?includeOutOfStock=1`) instead of disappearing or sitting mixed into the main grid — the category filter tabs now apply to both sections.
21. **A vendor-visibility contradiction was found and fixed.** Four different places computed "is this vendor's store visible/purchasable" differently — the vendor directory, single-vendor lookup, and product browse/detail routes all let a *suspended-but-KYC-verified* vendor's storefront and products keep showing (`status = 'active' OR kyc verified`), while checkout (`POST /api/orders`) never had that bypass and the AI assistant's candidate search had no vendor-status check at all. Net effect: a suspended vendor's products could browse fine, get recommended by the assistant, and then fail at checkout — an inconsistent, confusing experience. Standardized to `status = 'active'` only, everywhere, matching checkout's already-strict rule.
22. **The KYC-required-to-list-products check now actually respects the "Require ID/business verification for new vendors" toggle.** `admin/settings.html`'s description for that toggle says "Turning this off skips that requirement," but `POST /api/products` required verified KYC unconditionally regardless of it — a vendor auto-approved while the toggle was off still couldn't list a single product. Fixed to only enforce the check when the toggle is on, same condition the vendor-approval gate already used.
23. **Suspend/reactivate (customer and vendor) now emails the account, not just an in-app notification** — same copy, sent alongside the existing `notify()` row, so the account holder finds out even if they never reopen the app to see it.
24. **Guest visitors landing on an account-only page (order history, profile/settings) now get a modal explaining they need to sign in, instead of an instant unexplained redirect to the sign-in page.** Dismissing the modal (Cancel/✕/overlay-click/Escape — the shared `CustomerUI.confirm()` modal gained an `onCancel` callback for this) sends them back to the dashboard rather than stranding them on the now-empty account page.

25. **Super Admin/Moderator can now change another customer's or vendor's email or phone number — new.** Previously there was no way for an admin to fix another user's contact info at all (`PATCH /api/auth/me` only ever edits the caller's own profile) — a real gap for a locked-out account or a typo'd signup email. Email changes are OTP-gated: a code is emailed to the *new* address to prove control of it, and the admin (having gotten the code back through a support channel — there's no customer-facing redemption page) enters it in the same admin console to finalize; the *old* address gets an immediate heads-up email regardless of whether the change ever completes. Phone changes are a direct edit with a required reason and a notice email — there's no SMS integration in this codebase to OTP-gate a phone number the same way, so this is explicitly framed (in the admin UI copy) as an extreme-case/account-recovery tool, not a routine edit.
26. **Two smaller inconsistencies closed alongside the vendor-visibility fix:** admin sign-in (`POST /api/auth/admin-signin`) never checked `status === 'suspended'` the way `/signin` and `/google` both did, so a suspended admin could still get a valid token that their very next request would then reject — now checked the same way. `errorHandler.js` only read `err.status`, not `err.statusCode` — `checkid.js`'s identity-verification errors set the latter, so the intended 503/502/422 was silently collapsing into a generic 500; fixed to read both. `/api/auth/google` also picked up a rate limiter it was missing (every other anonymous-reachable auth route already had one).
27. **Real payments — ✅ built.** Checkout is a Paystack payment for the whole cart; orders exist only once it's paid; riders are paid by Paystack transfer the moment the buyer's code is entered and vendors when escrow releases (minus a commission a Super Admin sets); cancellations, unavailable items and returns refund the buyer's card. See `PAYMENTS.md`.
28. **KYC verification moved to IDCheck.ng** and gained BVN as an identity option. The NIN, BVN or driver's licence is matched against the vendor's legal name; the CAC number is looked up. See `BACKEND_GUIDE.md` §3 (`vendor_kyc`).

---

10. Design system reference (for anyone extending a page)

Shared building blocks used across customer, vendor, and admin (defined per-area in each `assets/style.css`, kept visually identical):

- **Layout:** `.app-shell` → `.sidebar` (desktop) + `.phone` (scroll container) → `header#app-header` → `main.page-shell` → `nav.bottom-nav` (mobile).
- **Cards/sections:** `.vendor-section` (generic card container, used well beyond vendor pages), `.stat-grid` / `.stat-card` (KPI tiles).
- **Tables (admin only):** `.table-wrap` + `table.data-table`, row actions via `.table-actions button` with semantic classes `.btn-view/.btn-suspend/.btn-activate/.btn-approve/.btn-reject`.
- **Status pills:** `.status-pill` (orders: pending/processing/shipped/out-for-delivery/completed/cancelled — the last two states added for order tracking), `.badge` (admin + vendor reports panel: active/suspended/pending/resolved/dismissed/open).

- **Forms:** `.form-grid` / `.form-group` / `.form-input` / `.form-textarea`, `.switch` (toggle), `.danger-zone` (destructive settings). `customer/assets/style.css` also defines a lighter `.field-group` (label + input/select/textarea) shared by `settings.html` and `store.html`'s review form.
- **Per-field inline edit** (item 32): `.field-view` (read-only text + pencil button) / `.field-edit` (hidden until clicked — real input + green check/red x pair) — same class names and shape in all three apps' stylesheets (`vendor/profile.html`'s Store Details, `customer/settings.html`'s Profile card, `admin/settings.html`'s new Account Details card).
- **Modals:** `.modal-overlay` + `.modal-panel` (Add Product, confirm dialogs, detail views, vendor/orders.html's Update Shipment and Submit Evidence modals). `customer/assets/style.css` gained its own copy of this block for `orders.html`'s File a Report modal — same class names as vendor/admin, kept in its own file per the usual convention (the CSS stays duplicated per app, same as every other shared class; only the JS/markup generation for the confirm/info/photo-preview trio was consolidated). Vendor and customer both gained a `#confirm-modal` (item 34, `VendorUI` / `CustomerUI.confirm()` / `.info()`, mirroring `AdminUI`'s) and a `#photo-preview-modal` (item 36, `.photoPreview()` — `.photo-preview-frame.is-avatar` / `.is-cover` sizes the preview circular vs. wide) using this same `.modal-overlay` / `.modal-panel` shell. **As of item 40, none of the three apps' pages carry this markup in their own HTML anymore** — the project-root `shared-ui.js` (`VetraModal`) builds `#confirm-modal` / `#photo-preview-modal` itself and appends it to `document.body` the first time any page calls `.confirm()` / `.info()` / `.photoPreview()`, and each app's `assets/ui.js` (`AdminUI` / `VendorUI` / `CustomerUI`) is now a thin wrapper delegating to it, so every existing call site keeps working unchanged.
- **Filter tabs:** `.order-filter-tabs .filter-tab.active` — also reused as-is for `vendor/products.html`'s category filter tabs (item 35), since it's a generic pill-row layout, not order-specific.
- **Report cards:** `.report-list` / `.report-card` / `.report-card-head` / `.report-reason` / `.report-meta-row` / `.report-actions` — originally admin-only (`admin/assets/style.css`), the same class names/shapes now also exist in `vendor/assets/style.css` for the read-only vendor-side view, so a report renders identically in both places.

If you add a new page to any app, copy the closest existing sibling page's `<head>`/sidebar/header/bottom-nav markup rather than writing it from scratch — this is how every existing page in the codebase was built, and it's what keeps the three apps visually consistent.